

Crash Course of Deep Learning Systems

Foundation to Optimization

2025/07/06

@erix025

ZJUSCT

Your AI pair programmer

GitHub Copilot uses the OpenAI Codex to suggest code and entire functions in real-time, right from your editor.

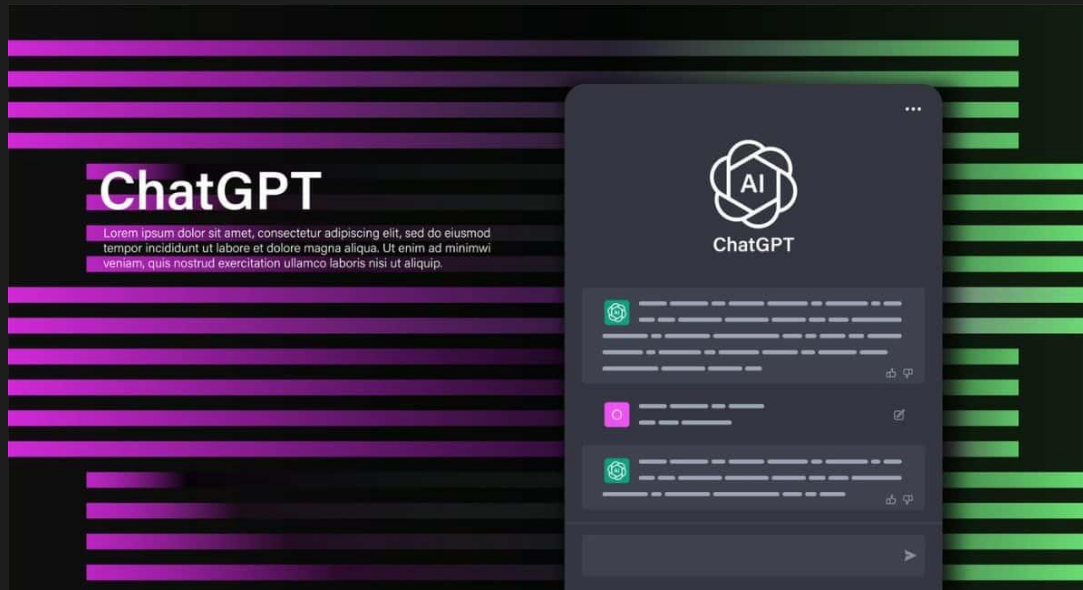
[Get Copilot for Business >](#)[Compare plans](#)

```
1 #!/usr/bin/env ts-node
2
3 import { fetch } from "fetch-h2";
4
5 // Determine whether the sentiment of text is positive
6 // Use a web service
7 async function isPositive(text: string): Promise<boolean> {
8   const response = await fetch("http://text-processing.com/api/sentiment/", {
9     method: "POST",
10    body: `text=${text}`,
11    headers: {
12      "Content-Type": "application/x-www-form-urlencoded",
13    },
14  });
15  const json = await response.json();
16  return json.label === "pos";
17 }
```

Replay

GitHub Copilot: Code Generation

ChatGPT



ChatGPT



Examples

"Explain quantum computing in simple terms" →

"Got any creative ideas for a 10 year old's birthday?" →

"How do I make an HTTP request in Javascript?" →



Capabilities

Remembers what user said earlier in the conversation

Allows user to provide follow-up corrections

Trained to decline inappropriate requests



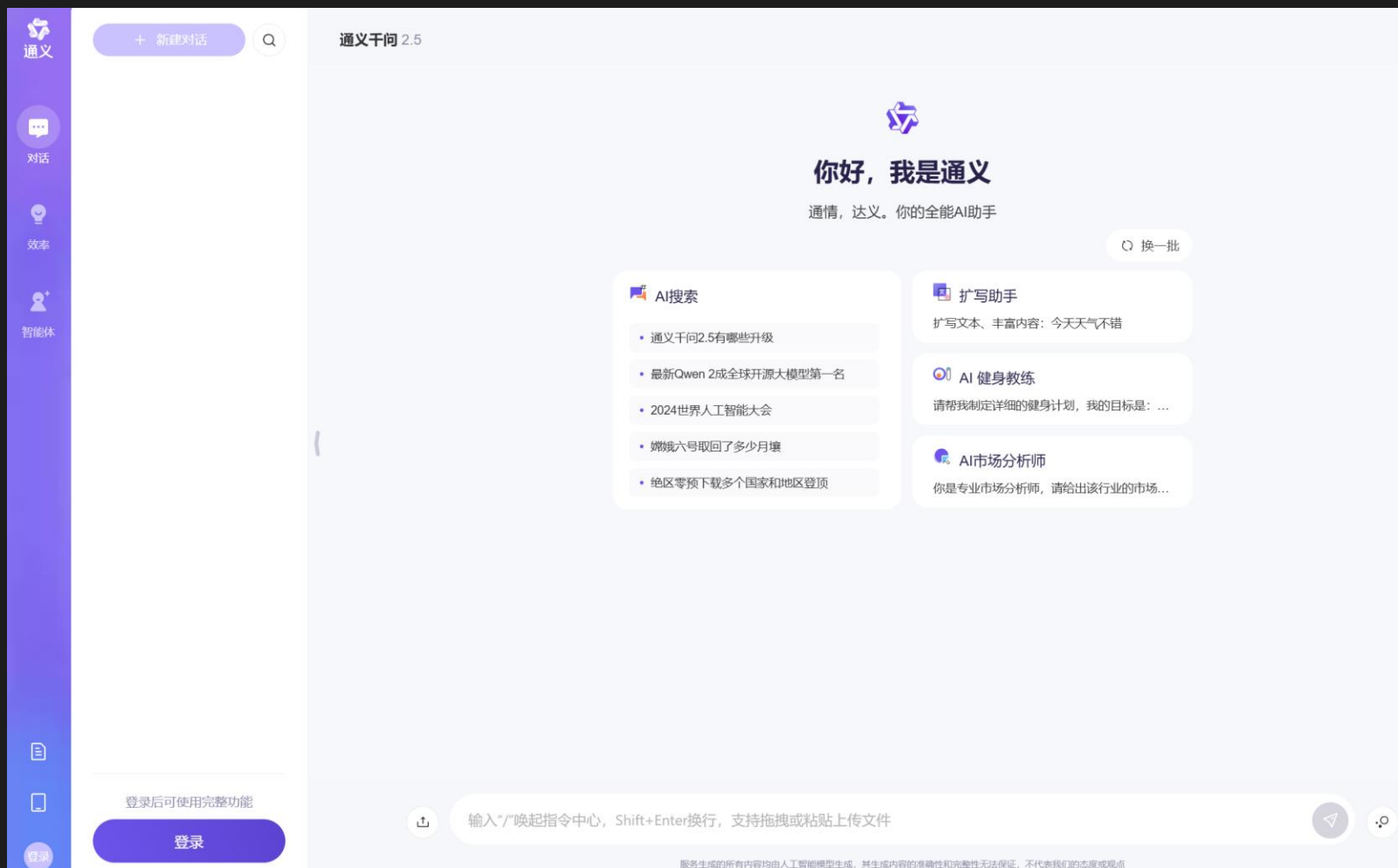
Limitations

May occasionally generate incorrect information

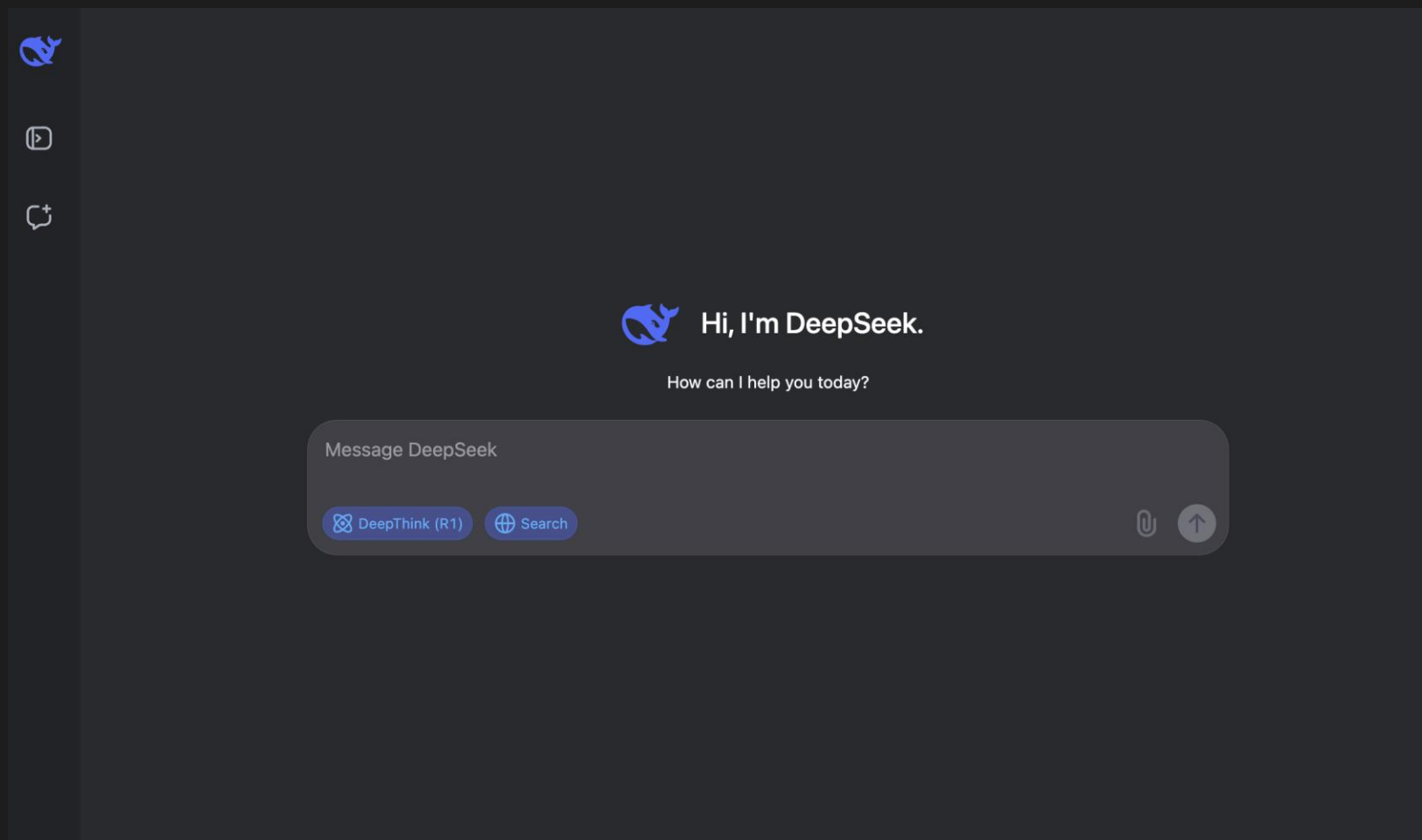
May occasionally produce harmful instructions or biased content

Limited knowledge of world and events after 2021

Qwen



DeepSeek



OF TEMPLES AND TOTEMS

by Razzleberries

32
FPS



NATIVE 4K

Path Tracing On, RTX 3080



89
FPS

up-sampling



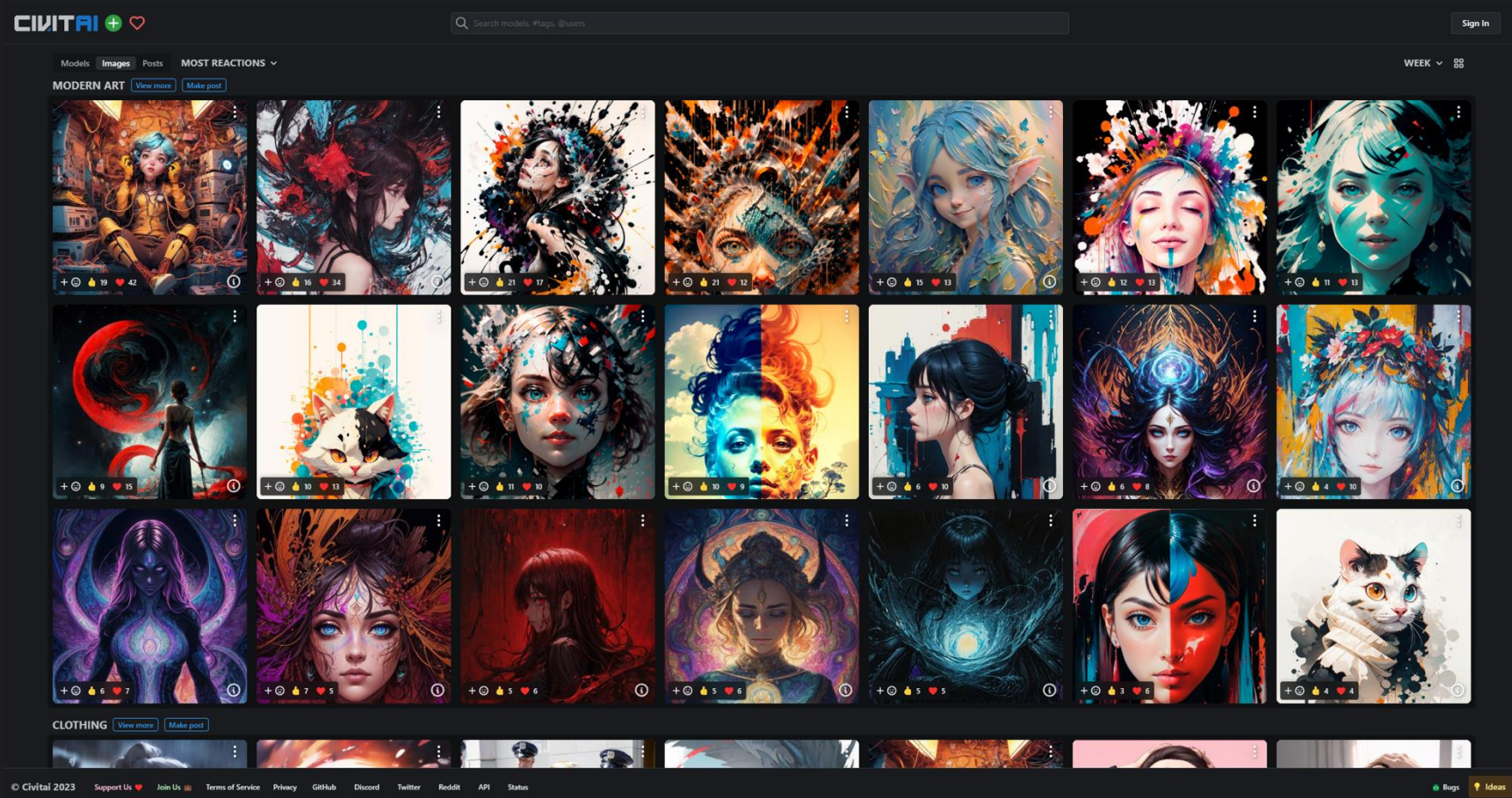
DLSS 4K

Path Tracing On, DLSS Performance, RTX 3080



AI Art

- Stable Diffusion + LoRA



AI Art

- Sora

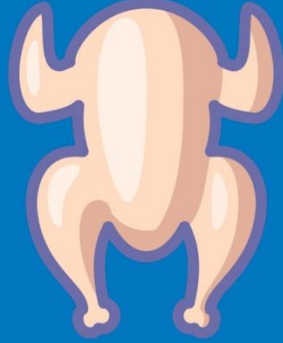


Traditional programming (software **1.0**)
(does not require 2.0)

VS.

(requires 1.0)
Machine learning (software **2.0**)

Traditional programming



1. Cut vegetables
2. Season chicken
3. Preheat oven
4. Cook chicken for 30-minutes
5. Add vegetables

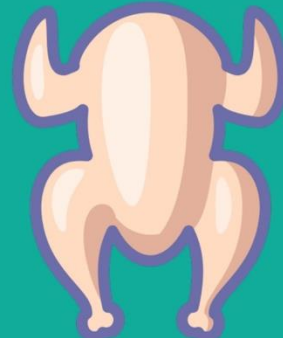


Starts with

Makes

Machine learning algorithm

Inputs



Output

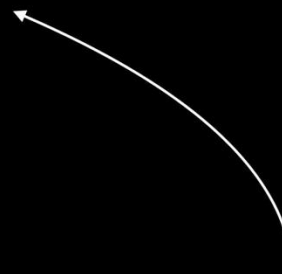


1. Cut vegetables
2. Season chicken
3. Preheat oven
4. Cook chicken for 30-minutes
5. Add vegetables

Starts with

Figures out

Machine learning is turning things (data)
into numbers and **finding patterns** in
those numbers.



The computer does this part.
How?
Math.
(we'll cover a little on this later)

“Why use machine learning?”

Another curious (perhaps even more curious than before) internet dweller

Good reason: ~~Why not?~~

Better reason: Can you think of
all the rules?

(probably not)

[illegible]



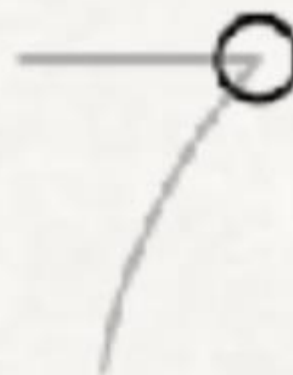
digit 7

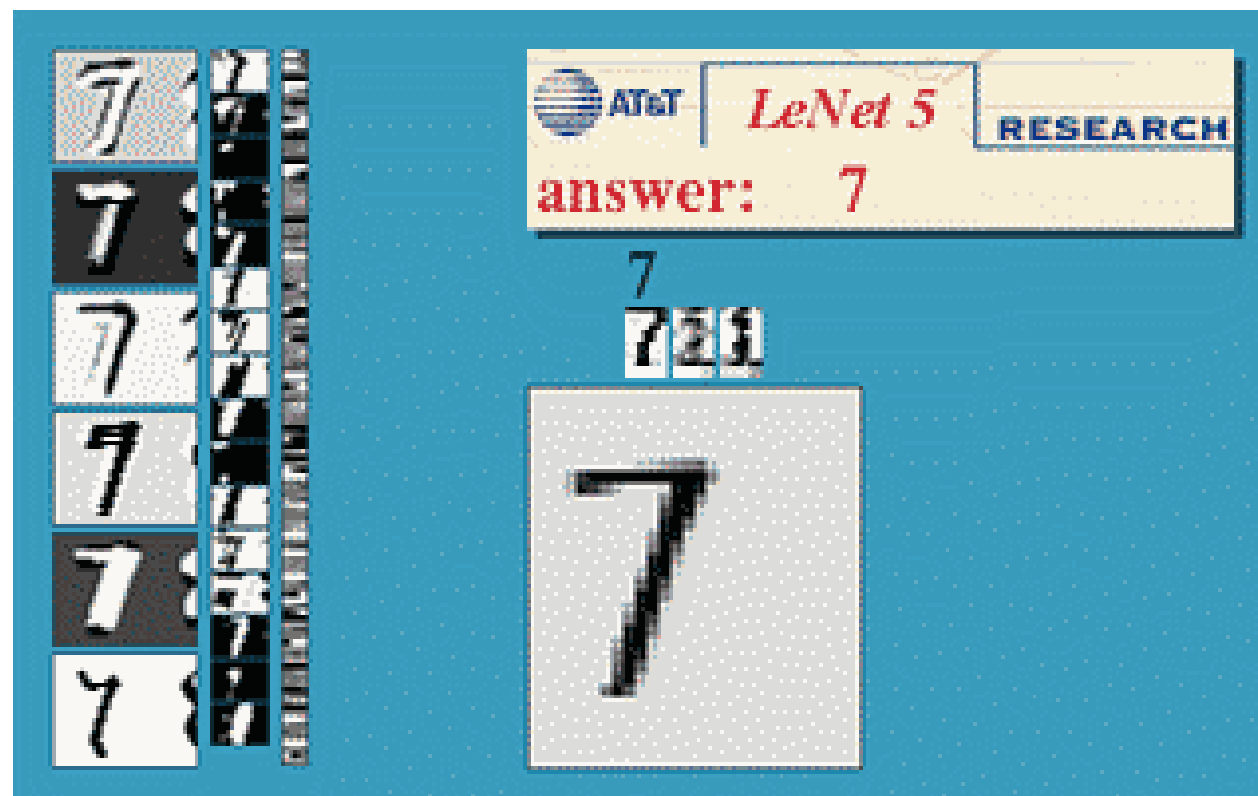
horizontal
line

+

NE-SW
diagonal

+

lines meet
at upper right



Traditional Approach vs Machine Learning

Traditional

- Hand-crafted rules / features
- Explainable
- Less tested on data
- Insufficient capacity

Machine Learning

- Automatically-learned rules / features
- Less explainable
- Data-driven
- High capacity

Focus on formulating the problem,
and then derive the solution more
automatically

(maybe not very simple...)

“If you can build a **simple rule-based** system that doesn’t require machine learning, do that.”

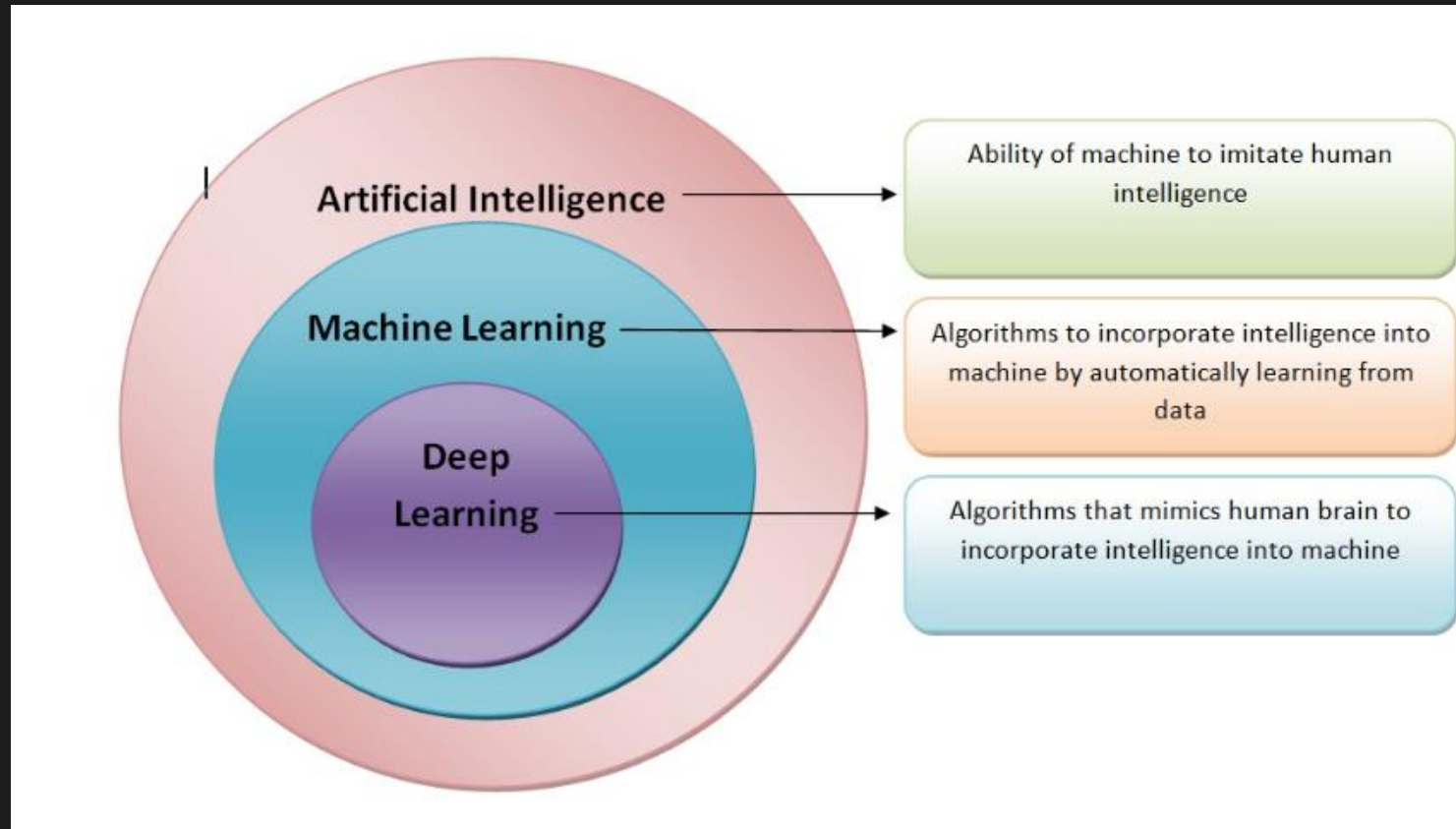
A wise software engineer... (actually rule 1 of Google’s Machine Learning Handbook)

What machine learning is good for



- **Problems with long lists of rules**—when the traditional approach fails, machine learning may help.
- **Continually changing environments**—machine learning can adapt (‘learn’) to new scenarios.
- **Discovering insights within large collections of data**—can you imagine trying to go through every transaction your (large) company has ever had by hand?

What is the difference between ML/DL/AI?



Why ML in an HPC course?

- ML (especially DL) is a special serious of application requiring large computing power
 - Just like other domain-specific applications
 - Can use techniques in traditional optimization
- ML can be used to guide system optimization
- Emerging research topic: **MLSys**
 - Distributed Learning
 - Inference Acceleration
 - Domain-Specific Language & Architecture
 - AutoSys
 - ...

What does this lesson focusing on?

- (Very) Basic ML & DL (What to do)
 - Model, Loss and Optimizer
 - CNN (CV)
 - Attention (NLP)
 - LLM
 - Framework (e.g. PyTorch)
- Techniques in Efficient AI
- Doesn't
 - Modern Deep Learning Technology (Many complex networks)
 - Theory (Why do this? Why it works?)
 - Application (How to use/deploy ChatGPT/Stable Diffusion?)

Contents

- Foundation of Deep Learning
- Topics in MLSys Optimization

Contents - Foundation of Deep Learning

- What is Deep Learning?
 - From Linear Regression to Neural Network
 - Training Neural Network
- What is inside in a Model?
 - Feed Forward Network (FFN)
 - Convolution Neural Network (CNN)
 - Recurrent Neural Network (RNN)
 - Transformer
 - Regularization

1. 🤔 Machine Learning Problems

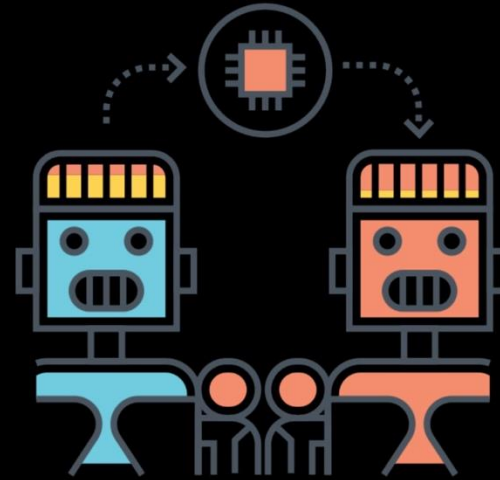
(categories of learning)



**Supervised
Learning**



**Unsupervised
Learning**



**Transfer
Learning**



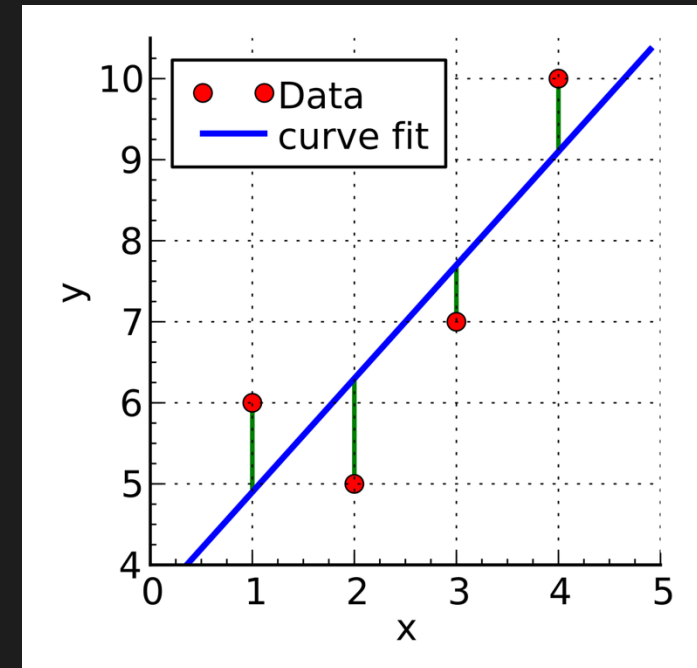
**Reinforcement
Learning**

From Linear Regression to Neural Network

Simple Linear Regression

- Model: $Y_t = ax_t + b$
- Loss Function: $L = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$
- Optimization: $L' = 0$

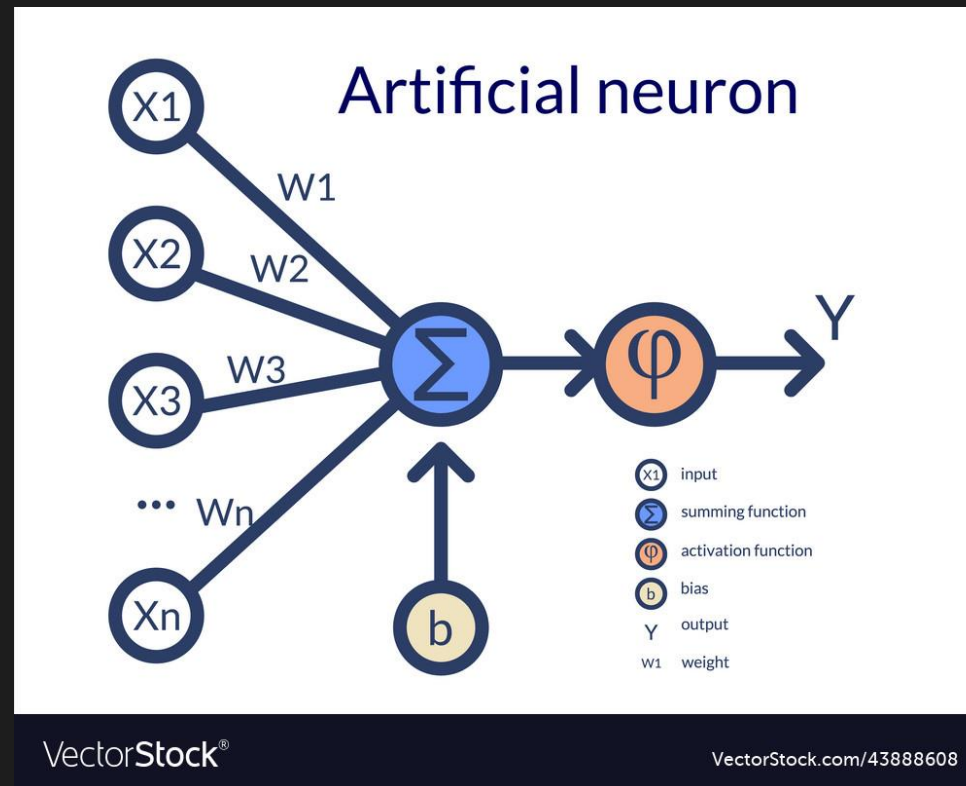
$$b = \frac{\sum Y_i}{n} - a \frac{\sum x_i}{n};$$
$$a = \frac{n \sum x_i Y_i - \sum x_i \sum Y_i}{n \sum x_i^2 - (\sum x_i)^2}.$$



From Linear Regression to Neural Network

Multiple Linear Regression

- Model: $y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i, \quad i = 1, \dots, n,$



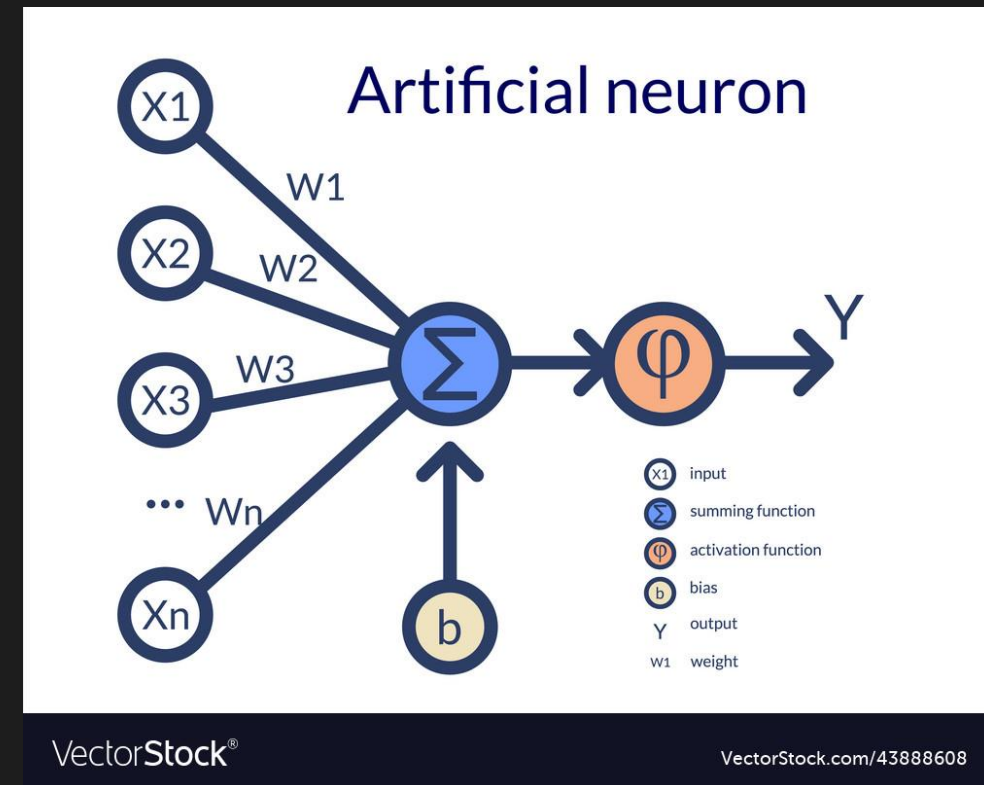
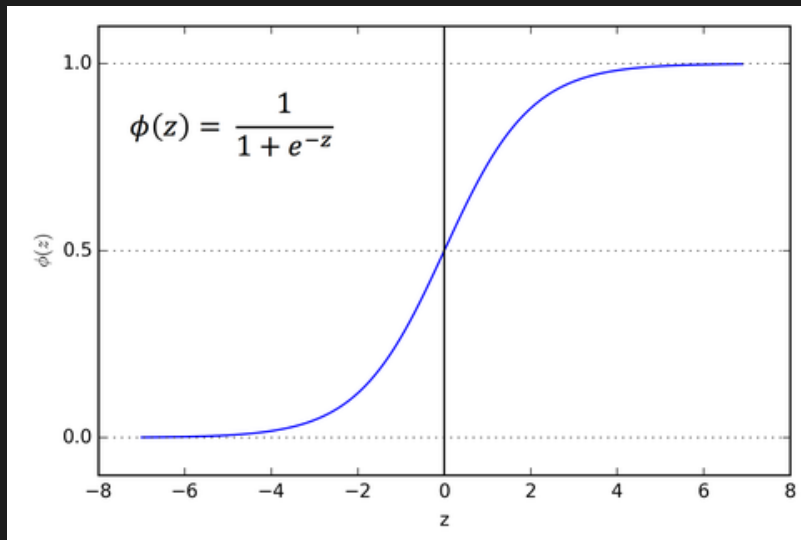
From Linear Regression to Neural Network

Non-Linear Regression: Activation Function

- Model:

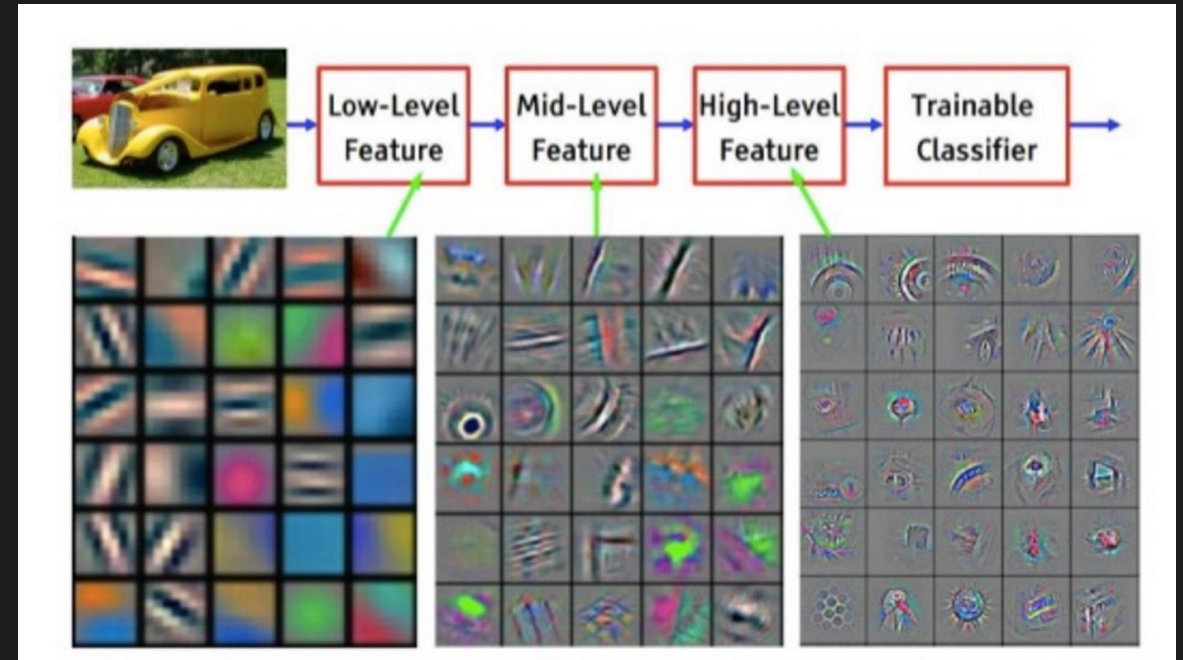
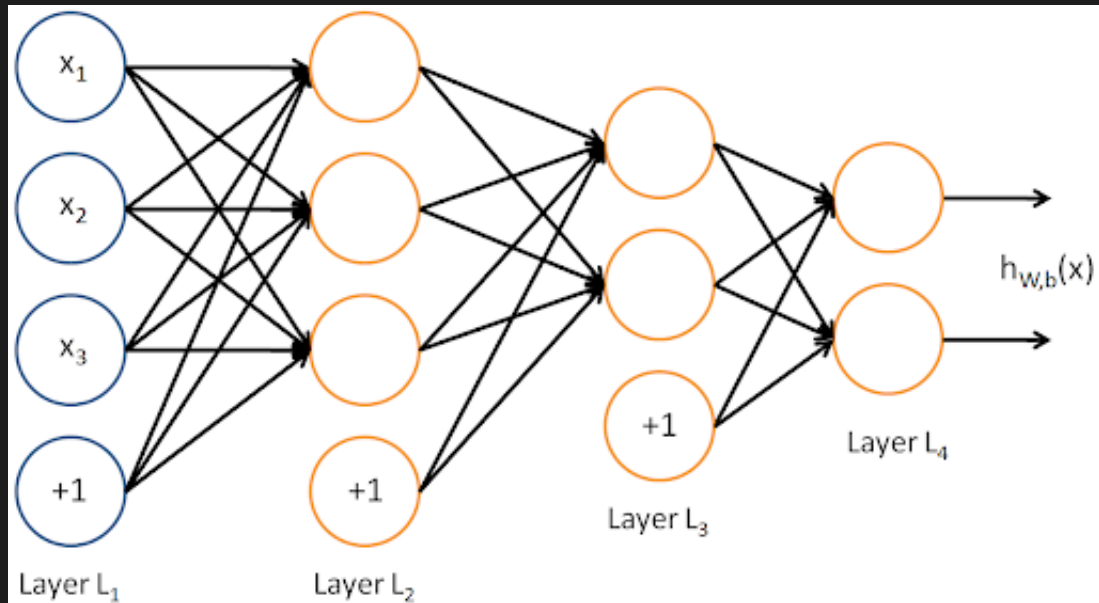
$$y = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-(w^T x + b)}} \quad , \quad y \in (0,1), z = w^T x + b$$

- Activation: Sigmoid Function



From Linear Regression to Neural Network

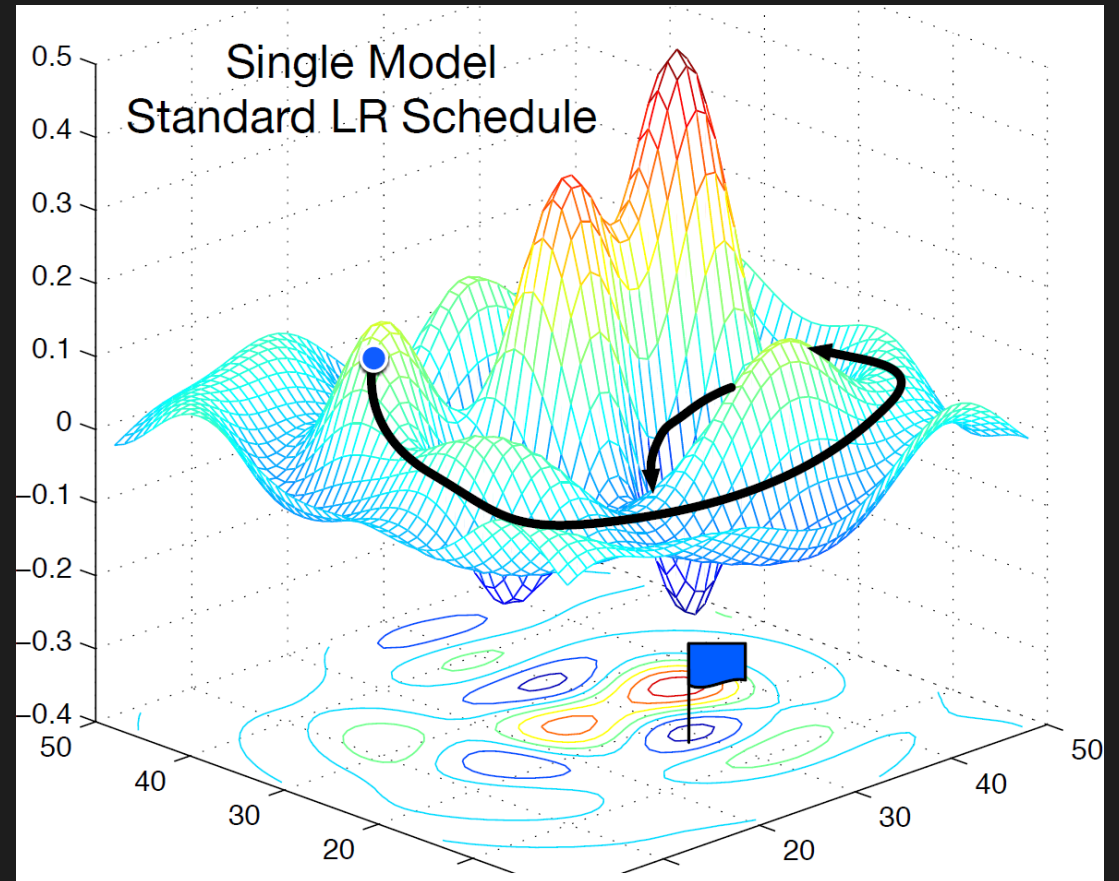
Single Layer -> Multiple Layers



Training Neural Network

Gradient Descent

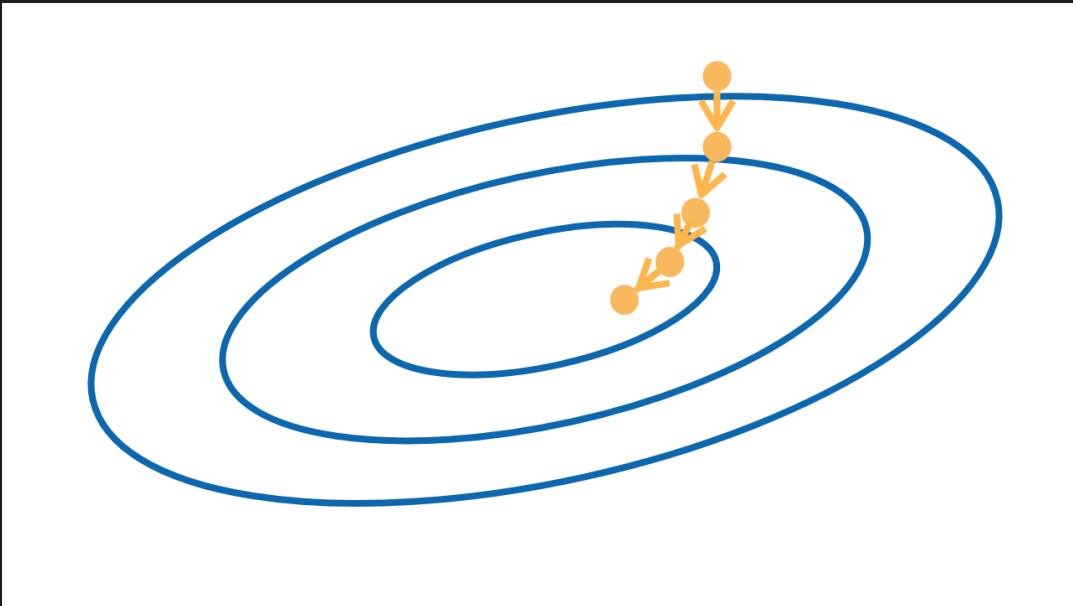
- So many parameters!
- How to solve (optimize): Gradient Descent
- $(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \eta \nabla L(\mathbf{w}, b)$
- Learning rate: η , length of one step
- Hyper-parameter: parameters whose values are used to control the learning process.
- Q: How to choose a learning rate?



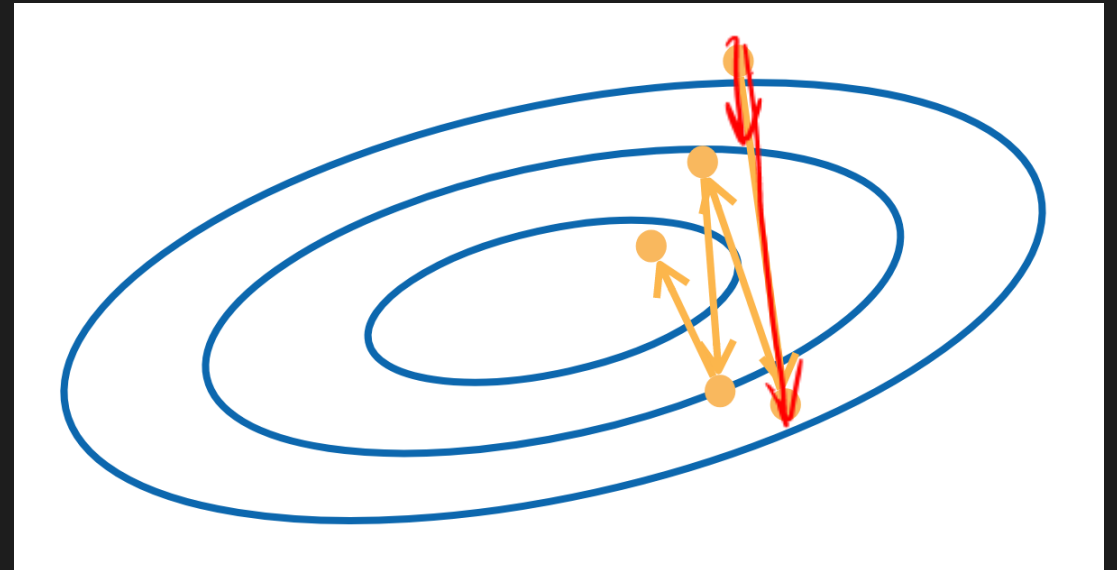
Training Neural Network

Choose a Learning Rate

Too small

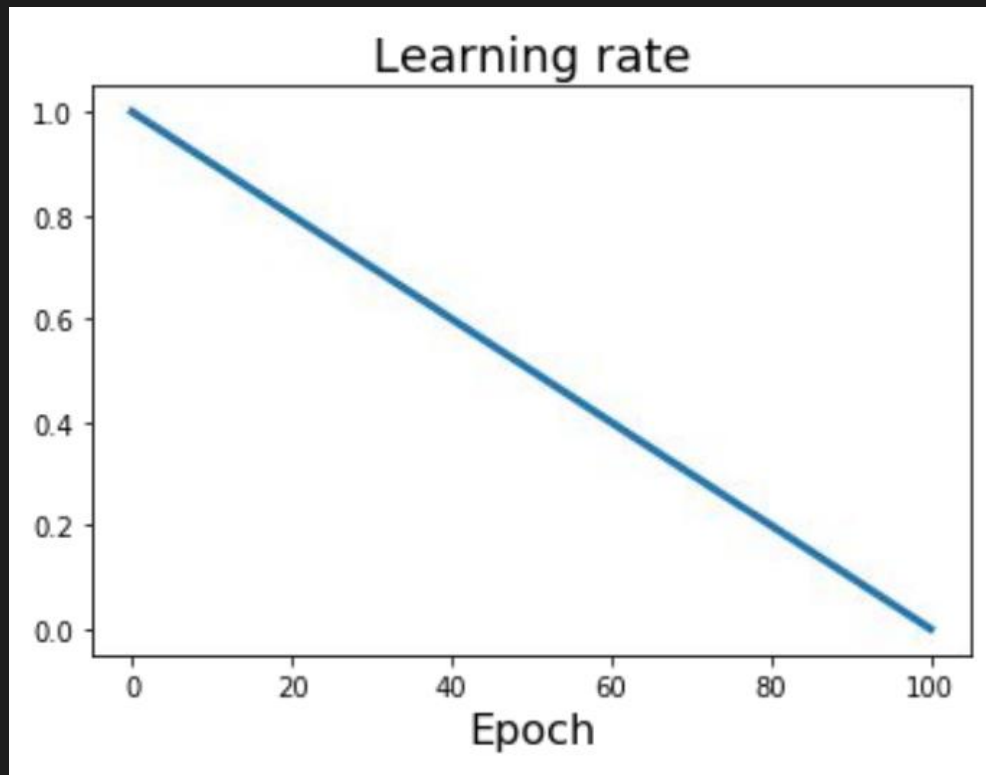


Too big

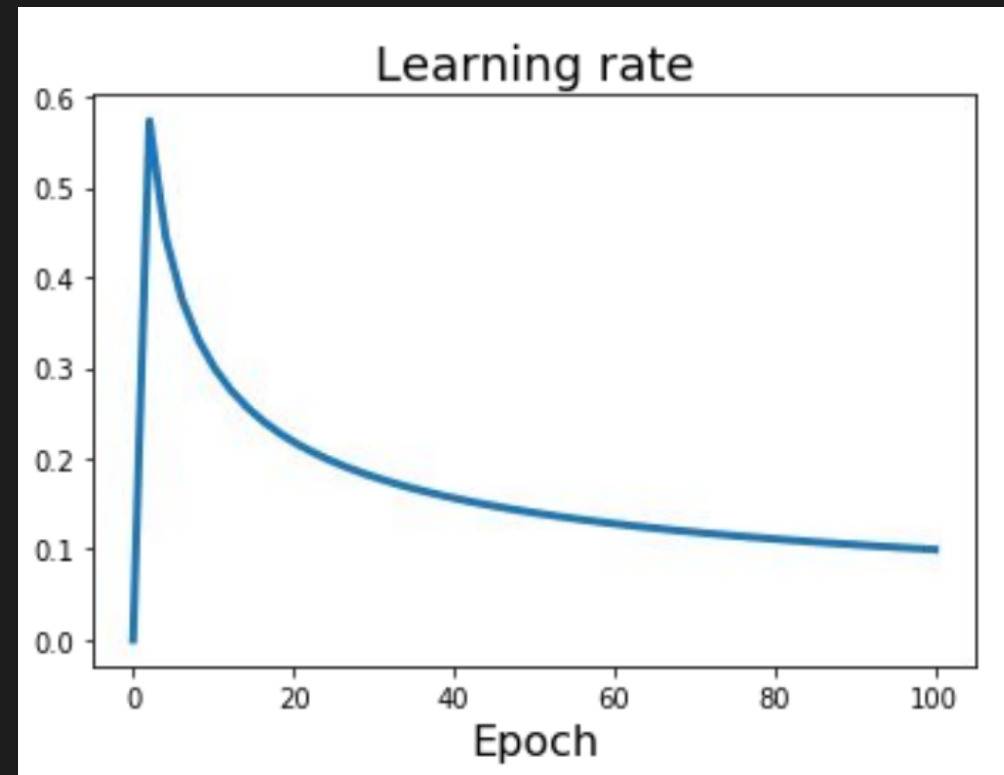


Training Neural Network

Learning Rate Scheduler



Linear Decay

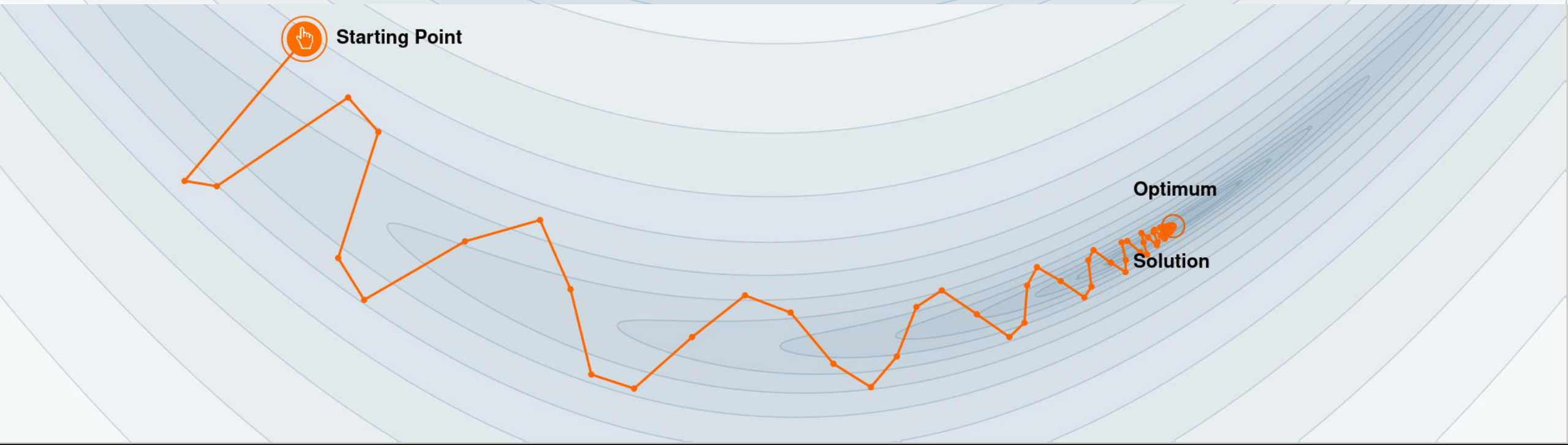
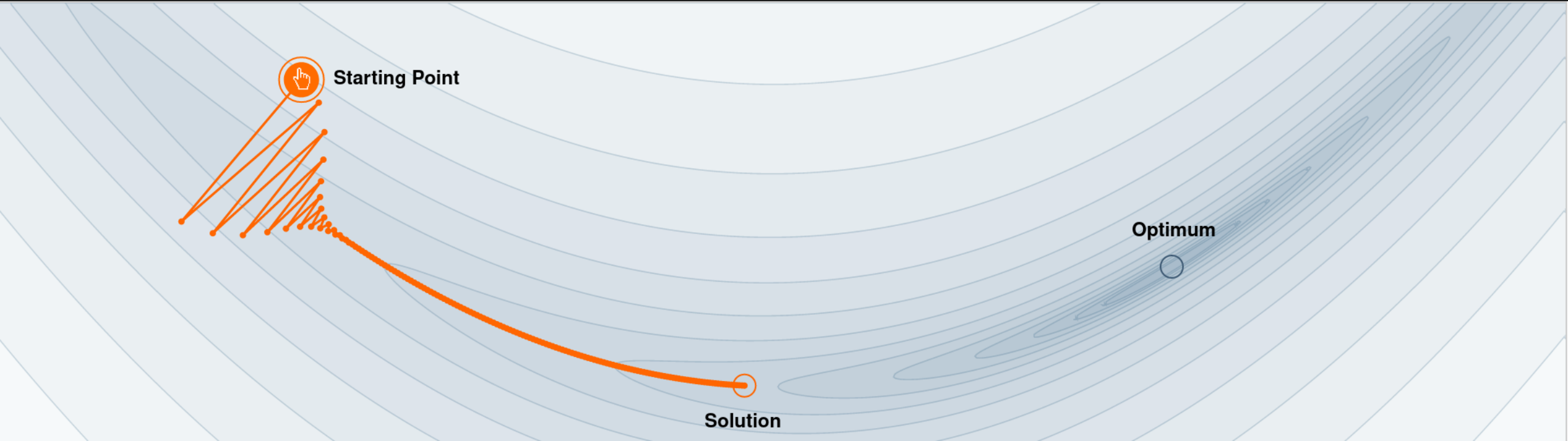


Linear Warmup

Training Neural Network

Better Optimizer

- **SGD:** $W \leftarrow W - \eta \nabla L(W)$
- **SGD + Momentum (Sutskever et al., 2013)**
 - $v \leftarrow \rho v + \nabla L(W)$
 - $W \leftarrow W - \eta v$
 - v : “velocity” as a running mean of gradients
 - ρ : “friction”, typically $\rho = 0.9$ or $\rho = 0.99$
- **Q: Why momentum?**



Training Neural Network

Better Optimizer

- **SGD:** $W \leftarrow W - \eta \nabla L(W)$
- **AdaGrad (Duchi et al., 2011)**
 - $s \leftarrow s + (\nabla L(W))^2$
 - $W \leftarrow W - \eta \frac{\nabla L(W)}{\sqrt{s} + \epsilon}$
 - Intrinsic: $\eta \leftarrow \eta / \sqrt{\sum_0^{t-1} (g_i^2)}$
 - Progress along “steep” directions is damped
 - Progress along “flat” directions is accelerated
 - Step size decays to zero over long time 🙄

Training Neural Network

Better Optimizer

- **SGD:** $W \leftarrow W - \eta \nabla L(W)$
-  **Adam (Kingma et al., 2015)**
 - At each iteration $i = 1, \dots, n$:
 - $v \leftarrow \beta_1 v + (1 - \beta_1) \nabla L(W)$
 - $s \leftarrow \beta_2 s + (1 - \beta_2) (\nabla L(W))^2$
 - **Bias correction:** $v' = \frac{v}{1 - \beta_1^i}, s' = \frac{s}{1 - \beta_2^i}$
 - $W \leftarrow W - \eta \frac{v'}{\sqrt{s'} + \epsilon}$
 - Adam with $\beta_1 = 0.9, \beta_2 = 0.999$, and $\eta = 10^{-3}$ or 5×10^{-4} is a great starting point for many models! 😎

Handwritten notes in red:

$$\beta_1 = 0.9 \quad V_0 = 0$$

$$V_1 = 0 \times 0.9 + \nabla L(w) \times \alpha$$

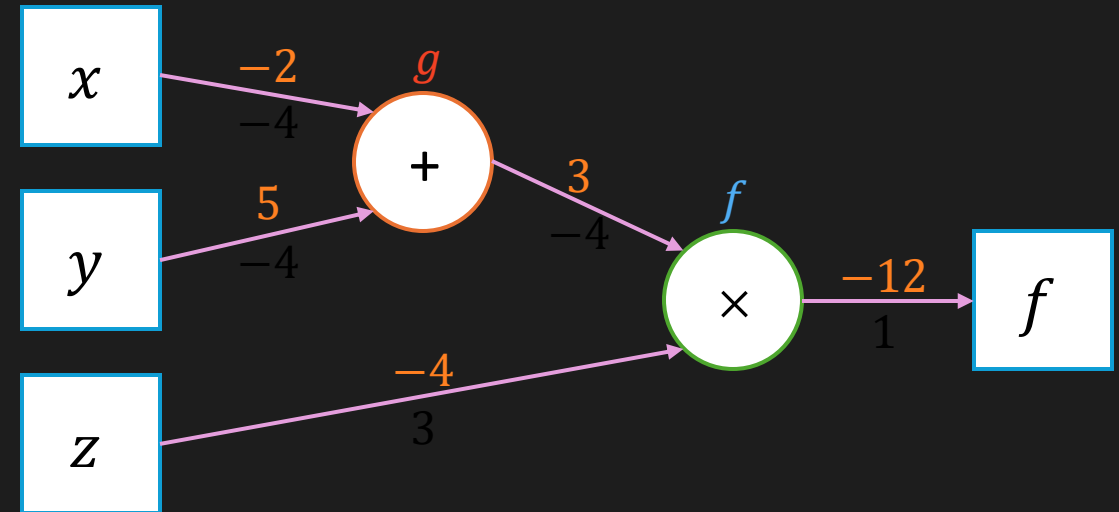
$$V_i = \frac{V_1}{1 - \beta_1} = \frac{0.1 \nabla L(w)}{1 - 0.9} = \nabla L(w)$$

Training Neural Network

Backpropagation

- How to calculate gradient?
 - Calculate Manually: Costly and inflexible
 - **Backpropagation**

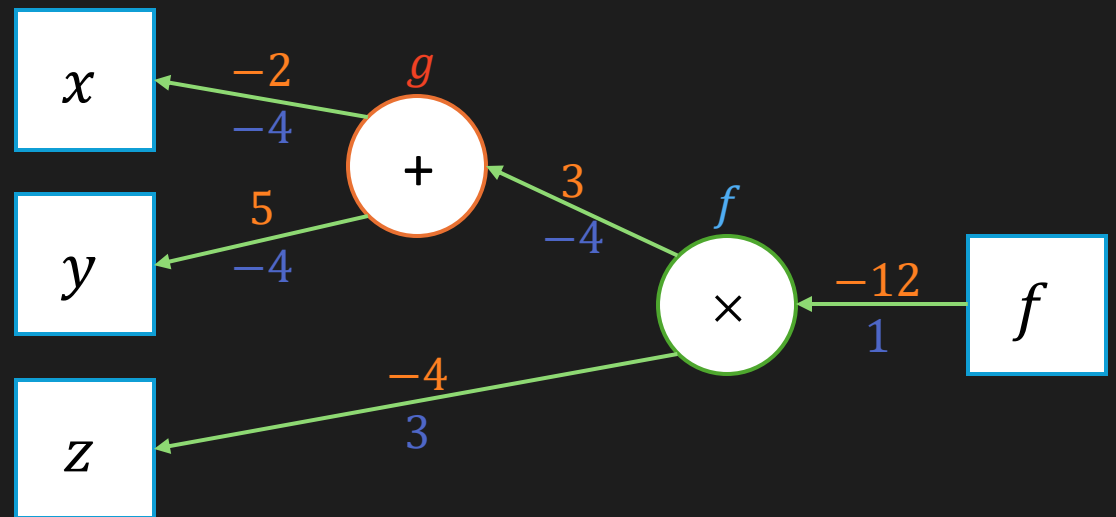
$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$



Training Neural Network

Backpropagation

- $f = f(x, y, z) = (x + y)z$
 - $g = x + y$
 - $f = gz$
- e.g. $x = -2, y = 5, z = -4$
- Find $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$
- $\frac{\partial f}{\partial f} = 1$
- $\frac{\partial f}{\partial g} = z = -4, \frac{\partial f}{\partial z} = g = 3$
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x} = -4, \frac{\partial f}{\partial y} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial y} = -4$



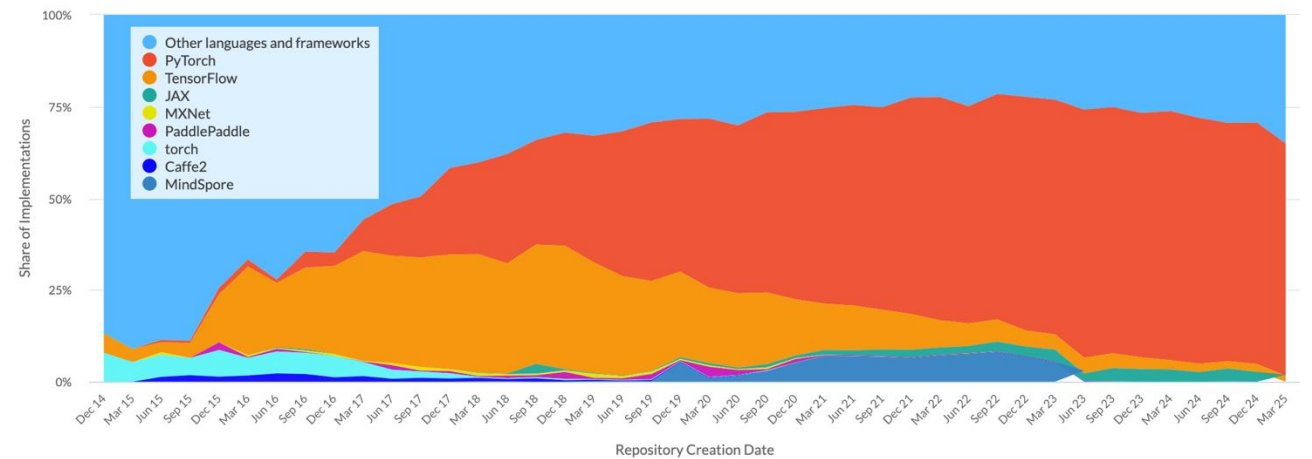
Training Neural Network

Deep Learning Frameworks

- PyTorch
- TensorFlow
- JAX
- MindSpore

Frameworks

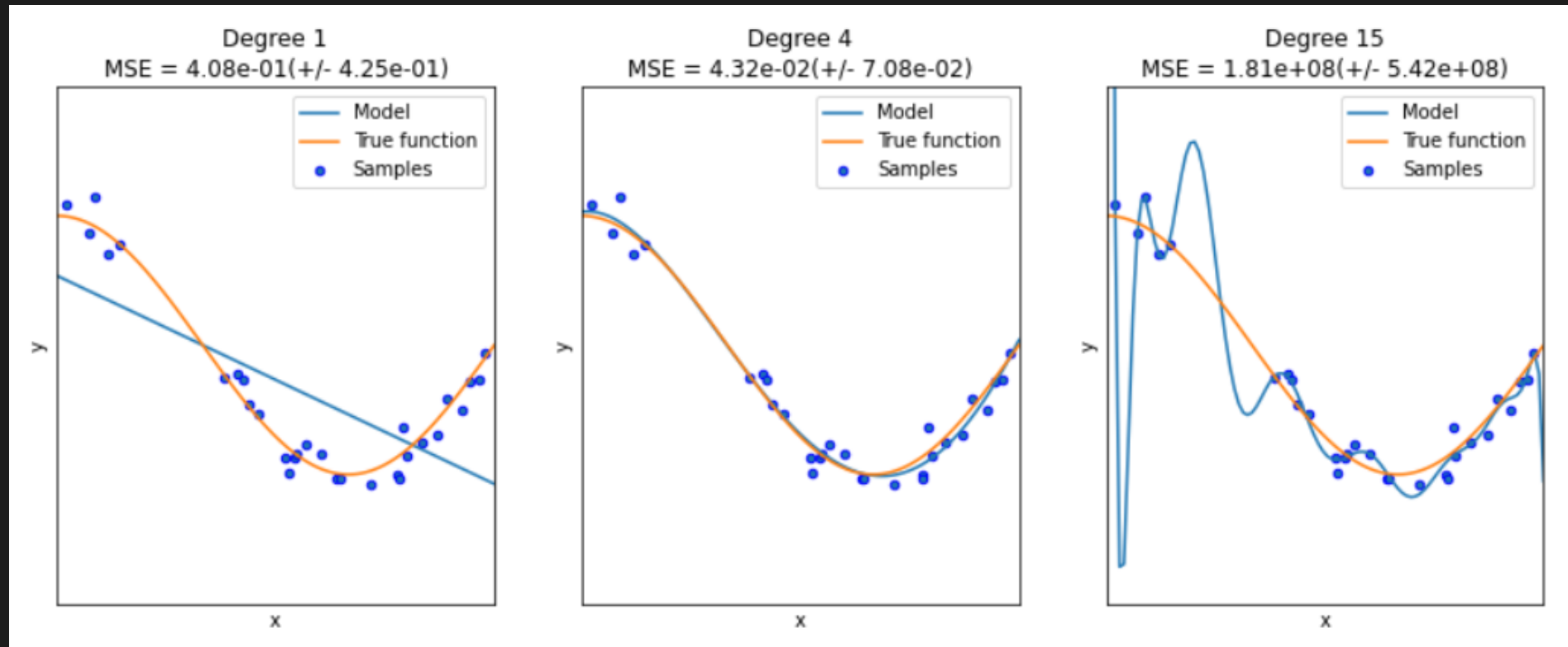
Paper Implementations grouped by framework



Code: How does these framework implemented?

Training Neural Network

Overfitting & Underfitting



Underfitting



Optimum

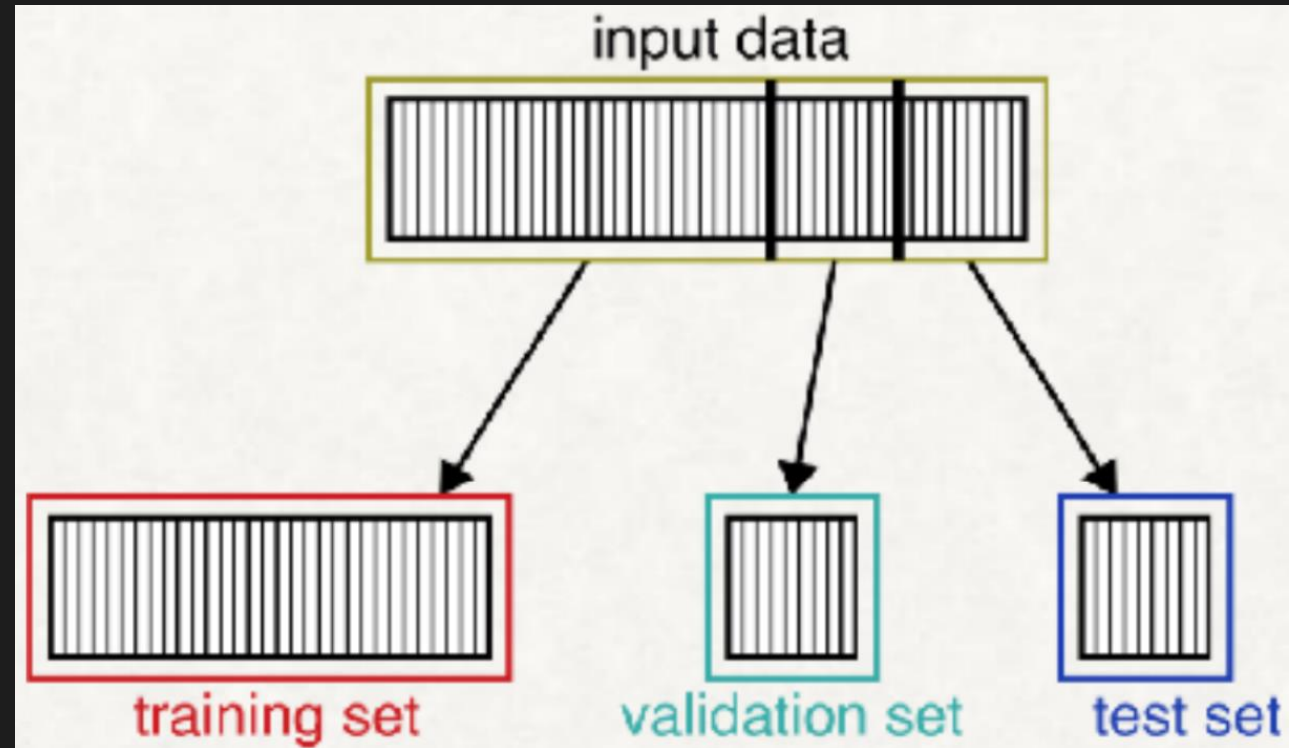
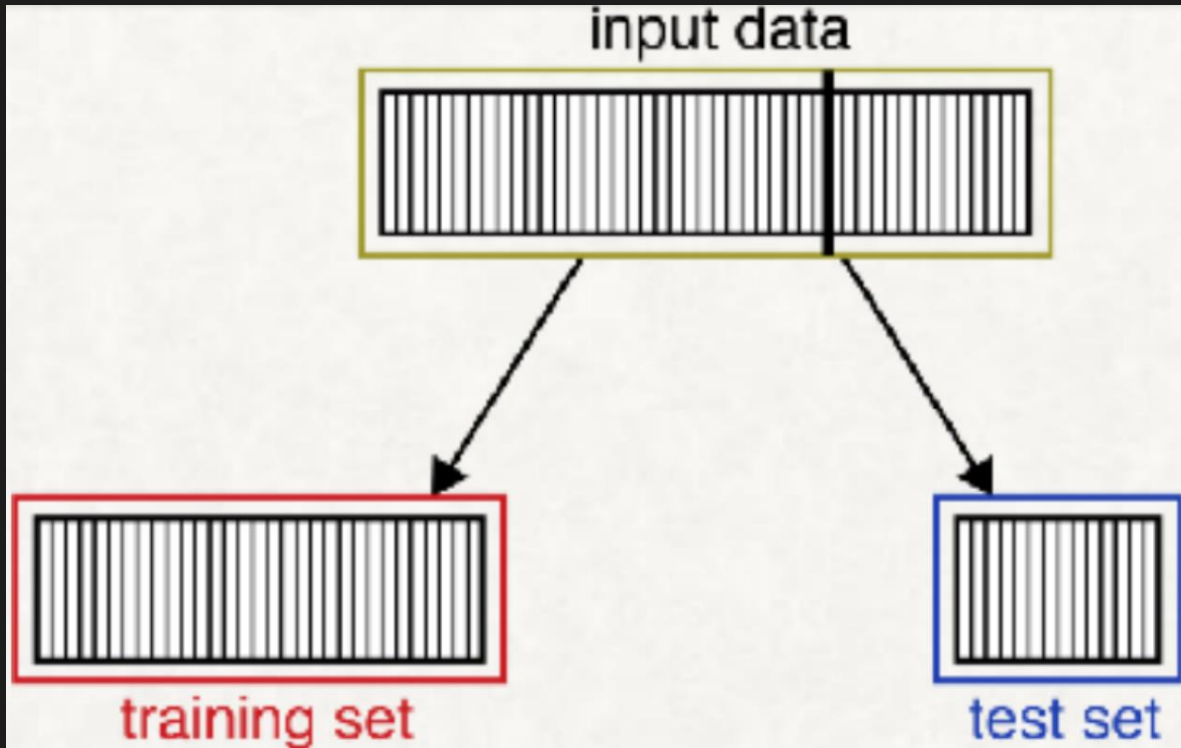


Overfitting



Training Neural Network

Training, Validation and Test Set



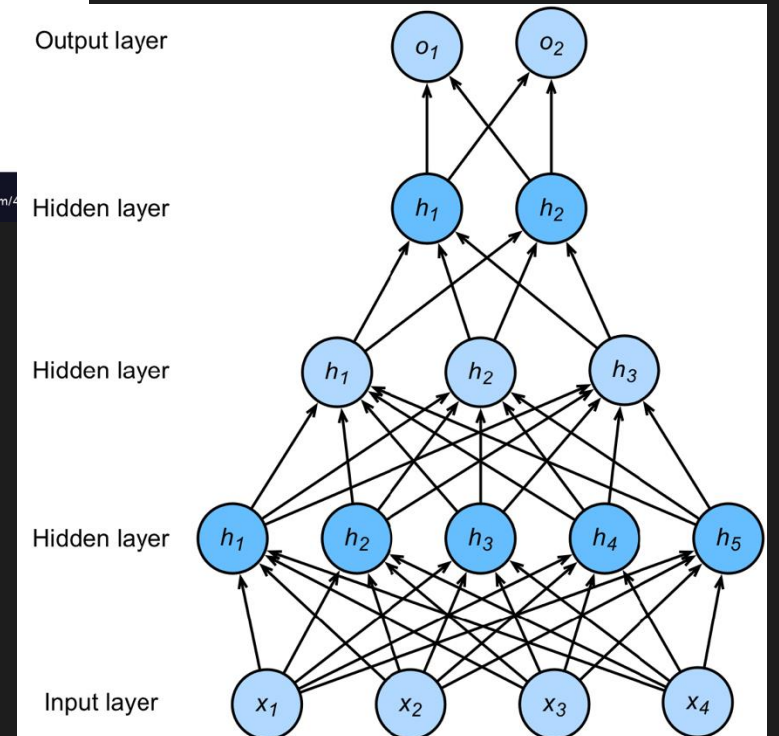
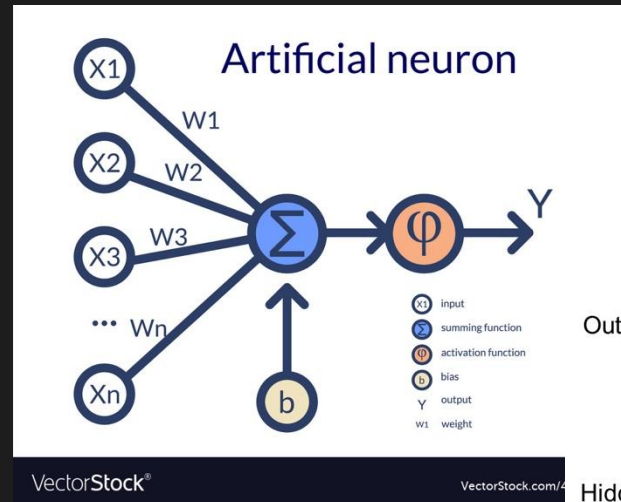
Any Questions?

Feed Forward Neural Network (FFN)

What is Inside in a Model?

Feedforward Network (FFN)

- Linear Layer
 - $y = xW + b$
 - Also known as linear projection or fully-connected layer
- Configuration
 - Input channel & output channel
 - Activation Function
- Parameters: Weight and Bias
- FFN: construct of Linear layers



What is Inside in a Model?

Activation Function

- Activation functions make MLP nonlinear

- w/o activation function:

- $\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$

- $\mathbf{o} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2 = \mathbf{W}_2 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} + \mathbf{b}' \rightarrow \text{linear...}$

- w/ activation function:

- $\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$

- What to use for σ ?

$$\mathbf{W}_2 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$
$$(\mathbf{W}_2 \mathbf{W}_1) \mathbf{x} + (\mathbf{b}_1 + \mathbf{b}_2)$$
$$\boxed{\mathbf{W} \mathbf{x} + \mathbf{b}'}$$

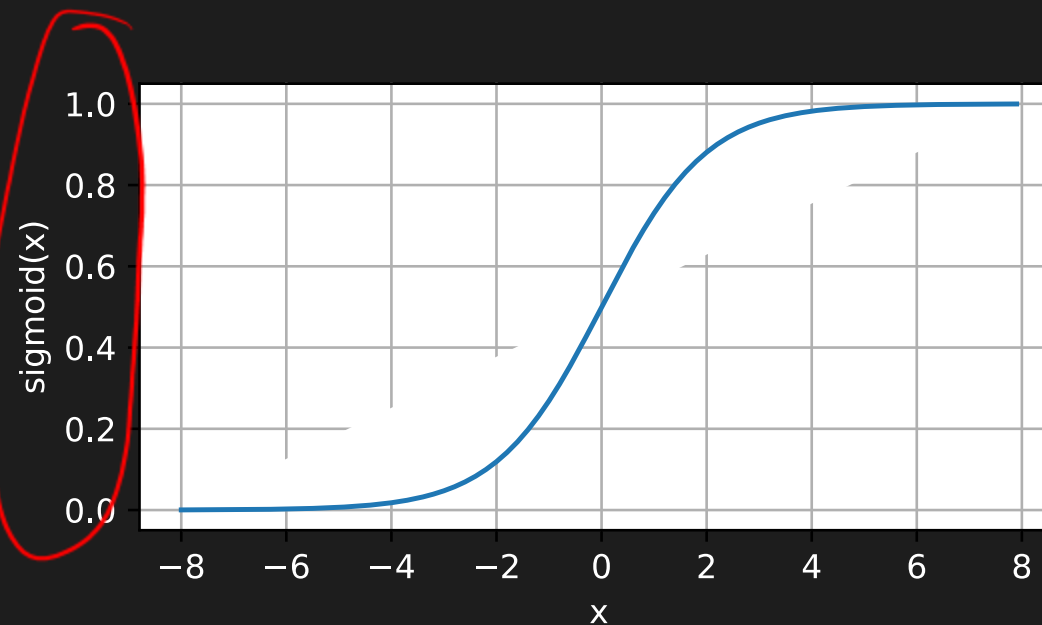
Sigmoid

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

- $\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$

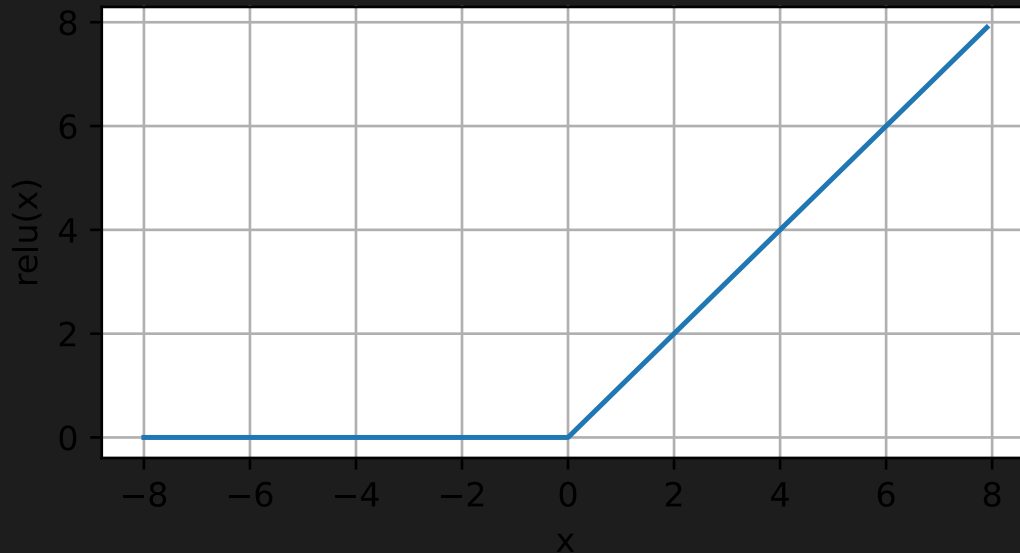
- $\frac{d}{dx} \text{sigmoid}(x)$
 $= \frac{\exp(-x)}{(1 + \exp(-x))^2}$
 $= \text{sigmoid}(x) (1 - \text{sigmoid}(x))$

(0,1)



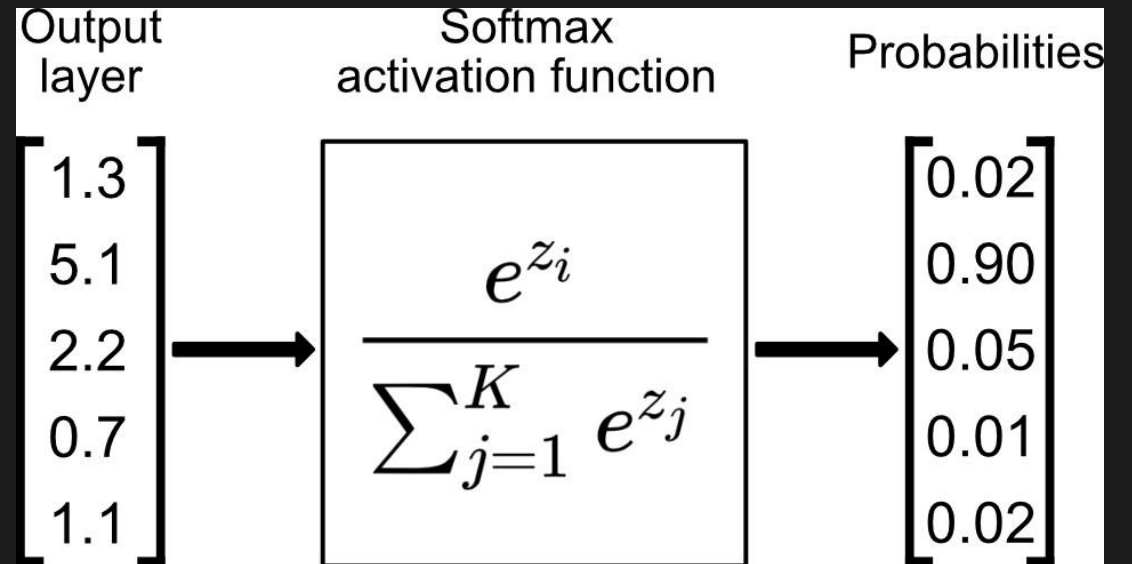
Rectified Linear Unit (ReLU)

$$\text{ReLU}(x) = \max(x, 0)$$



Softmax

- $\text{softmax}(\mathbf{o})_i = \frac{\exp(o_i)}{\sum_{j=1}^k \exp(o_j)}$
 - $i = 1, \dots, k$
 - $\mathbf{o} = (o_1, \dots, o_k) \in \mathbb{R}^k$



Convolution Neural Network (CNN)

Computer Vision Tasks

Classification



CAT

No spatial extent

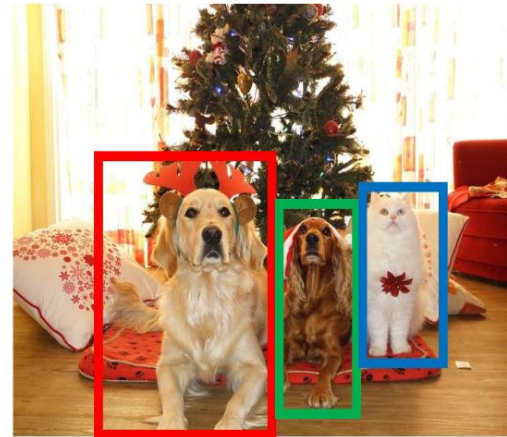
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

[This image is CC0 public domain](#)

What is Inside in a Model?

Convolutional Neural Network (CNN)

- Convolution

- $$y_{ij} = \sum_{m=0}^k \sum_{n=0}^k w_{mn} x_{i-\frac{k}{2}+m, j-\frac{k}{2}+n}$$

- Extract spatial features

- Configuration

- Kernel size (w, h)

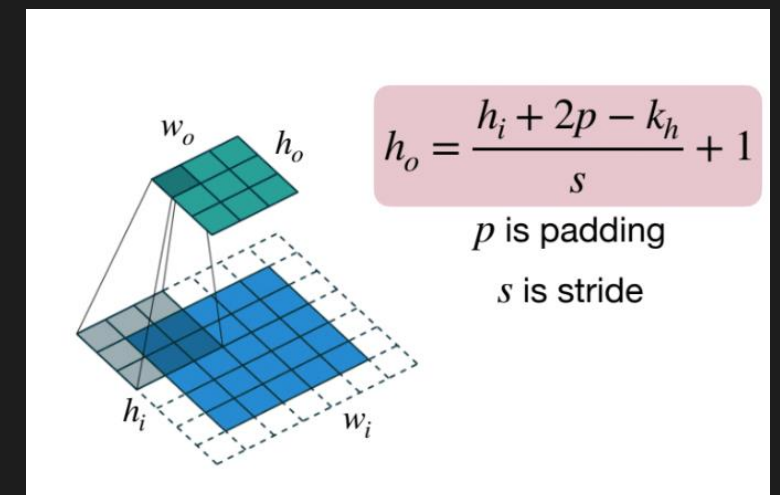
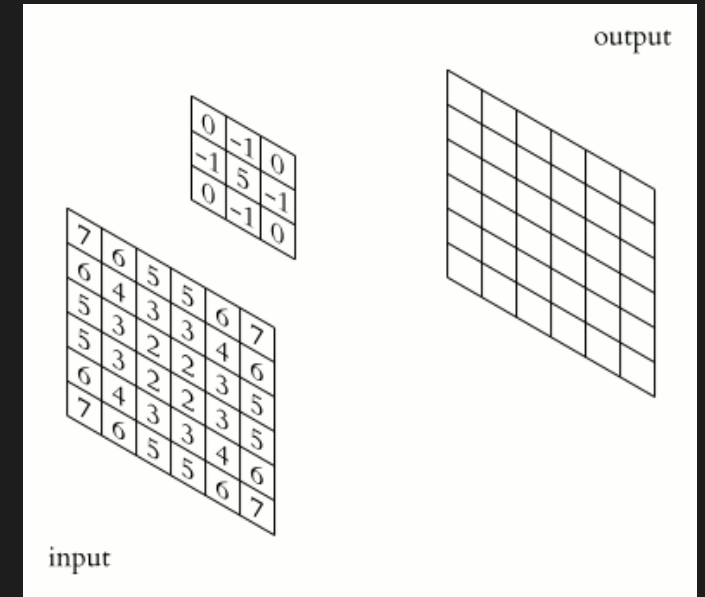
- Channel (c)

- Stride

- Padding

- Parameters: Kernel (w*h*c)

- Pooling



2D Discrete Convolution

$$(f * g)(i, j) = \sum_a \sum_b f(a, b) g(i - a, j - b)$$

Input

0	1	2
3	4	5
6	7	8

*

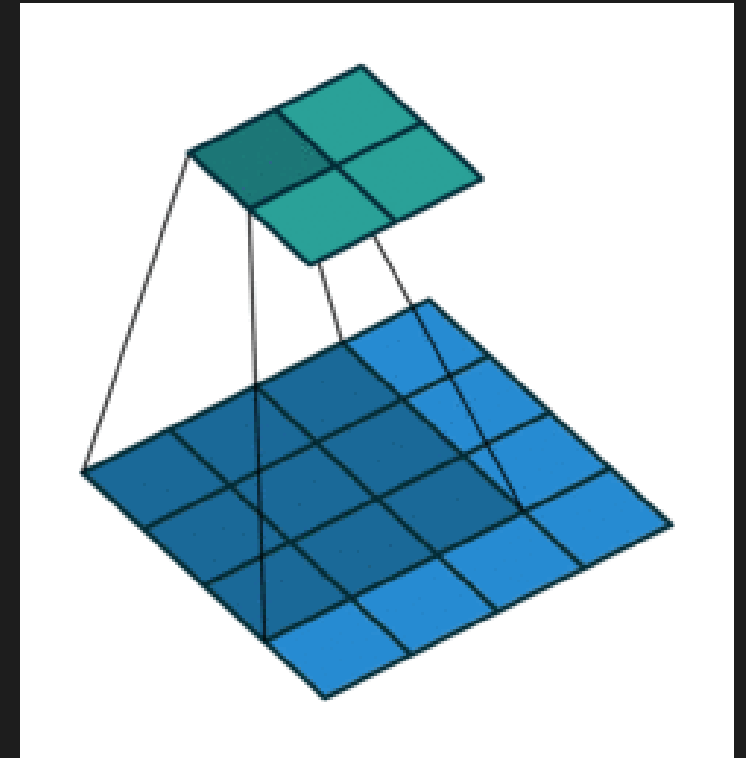
Kernel

0	1
2	3

=

Output

19	25
37	43



Padding

- Avoid losing pixels
 - Padding with 0
 - Padding with nearest neighbor

Input

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

(Padding = 1)

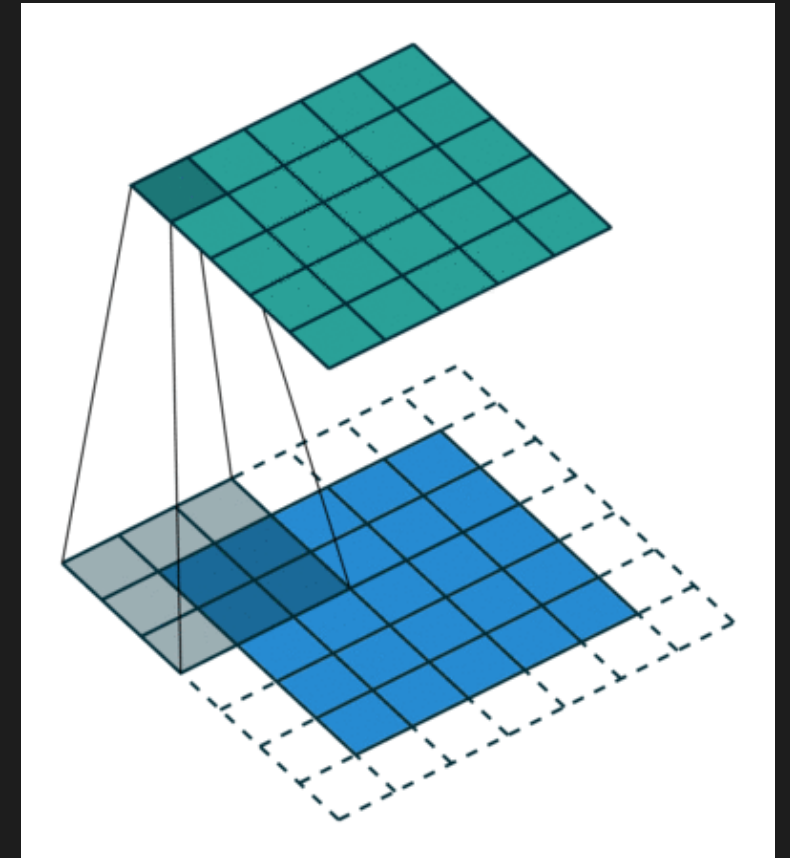
Kernel

0	1
2	3

Output

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0

* =



Stride

- Step length of movement of kernel

Input

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

(Stride x, y = 2, 3)

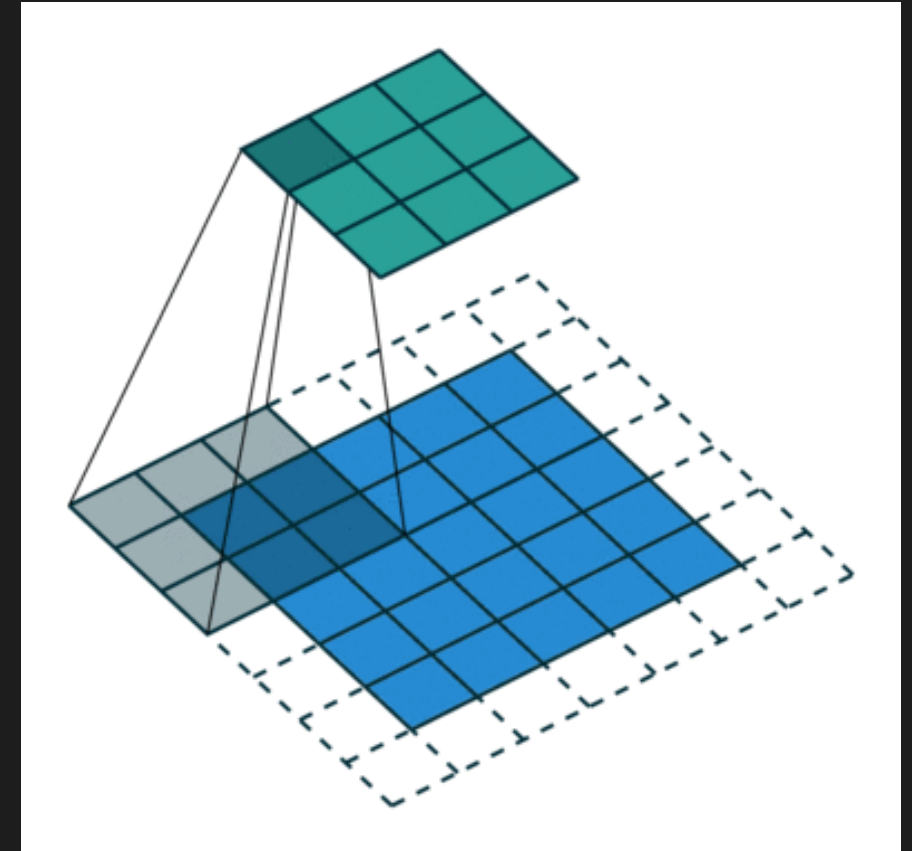
Kernel

0	1
2	3

Output

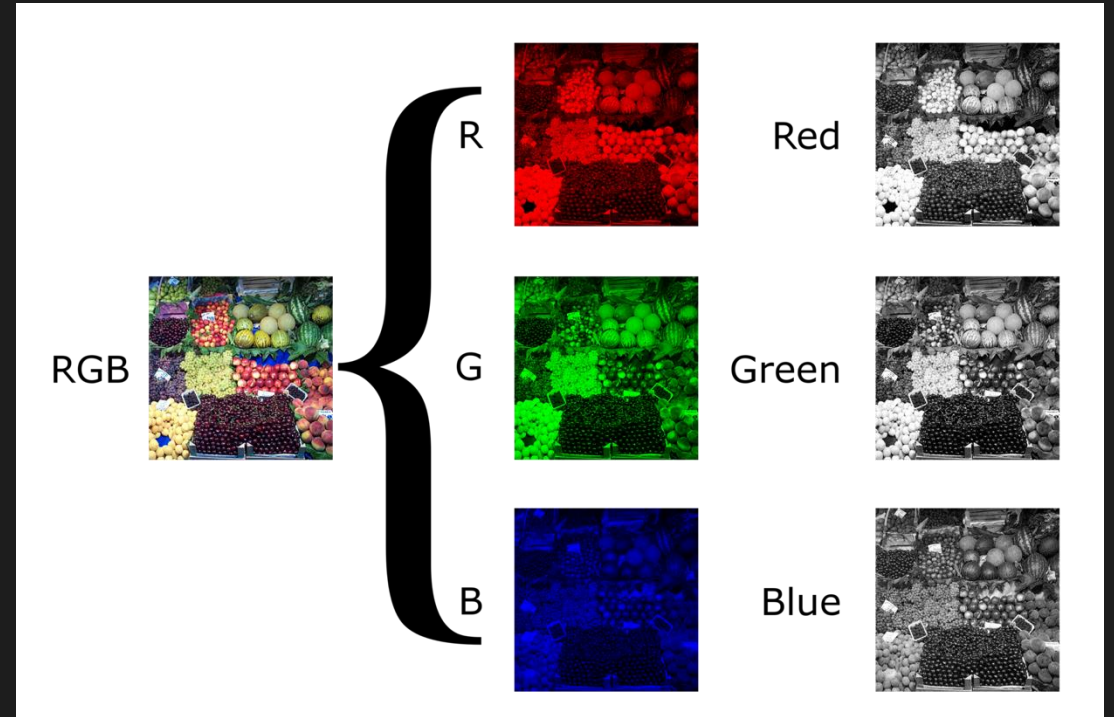
0	8
6	8

* =

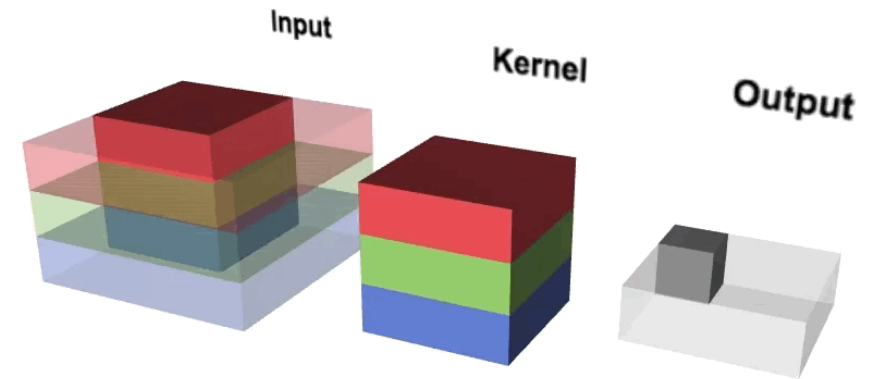
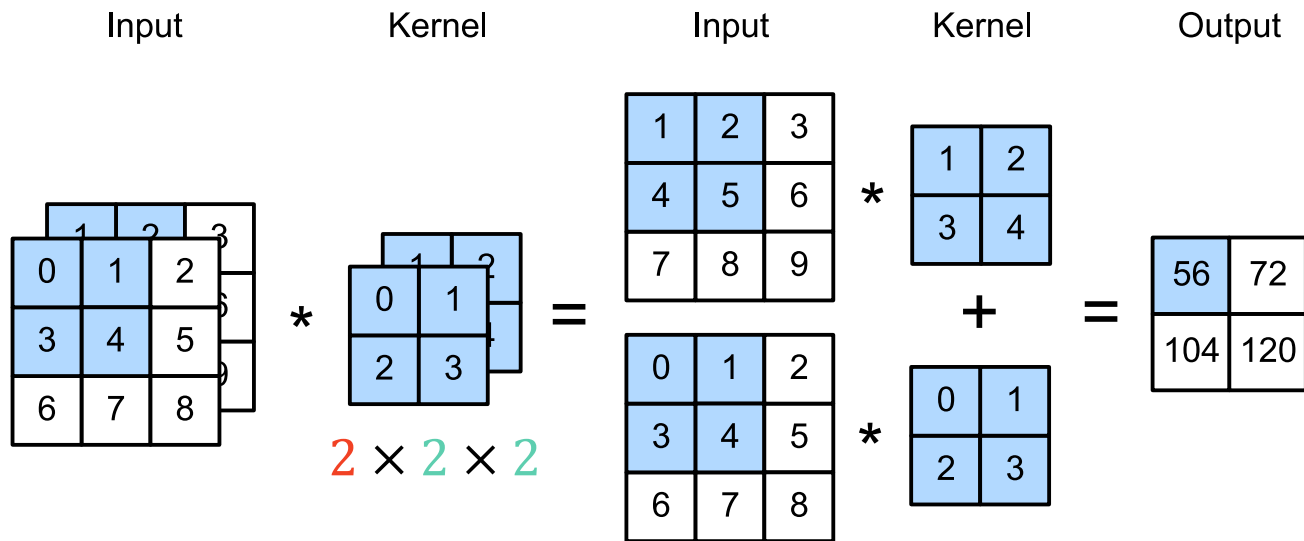


Multiple Input Channels

- Input data may contain multiple channels c_i .
- Kernel contains a tensor of shape $k_h \times k_w$ for every input channel.
- Concatenating these c_i tensors together yields a convolution kernel of shape $c_i \times k_h \times k_w$.



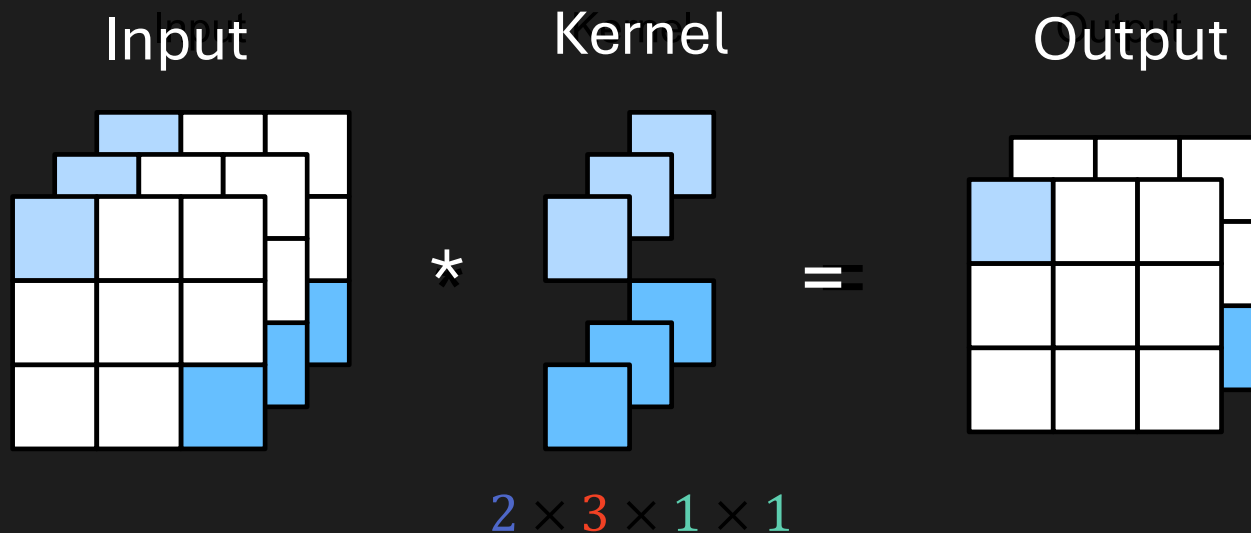
Multiple Input Channels



Multiple Output Channels

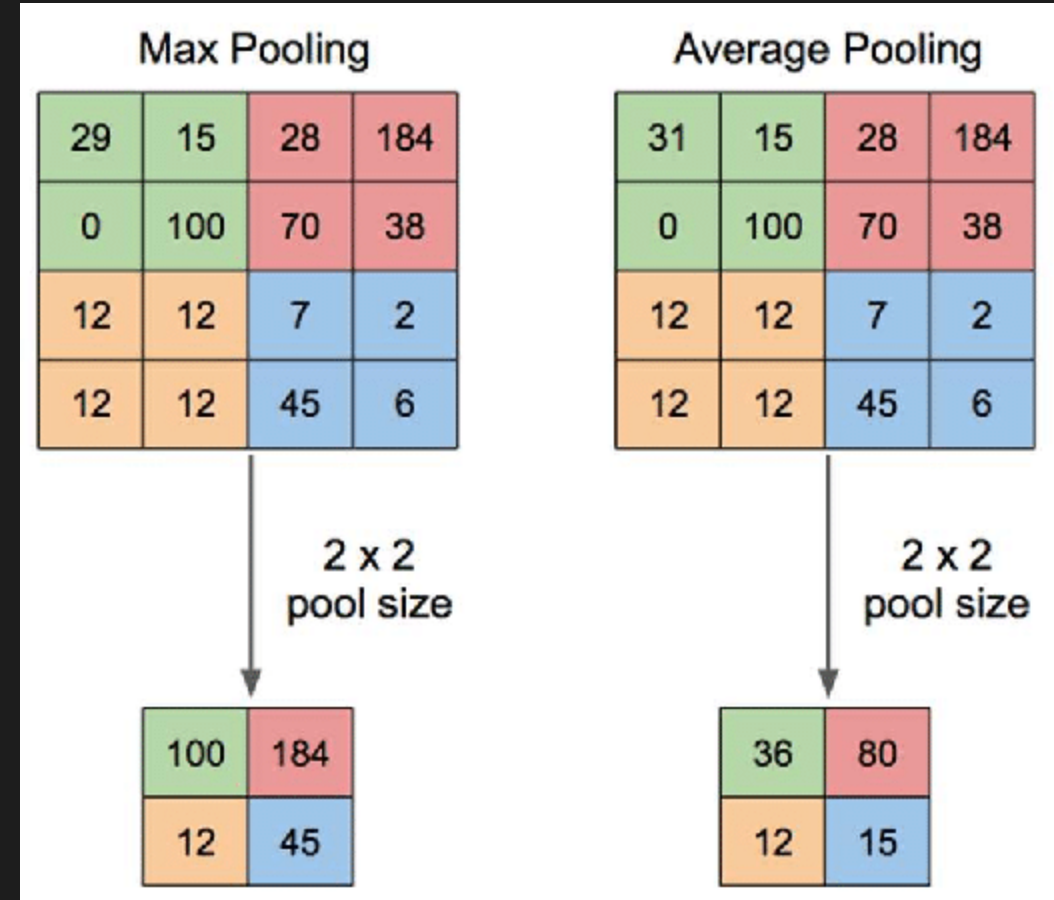
- Use c_o output different channels, each of shape $c_i \times k_h \times k_w$.
- Resulting shape of kernel:

$$c_o \times c_i \times k_h \times k_w$$

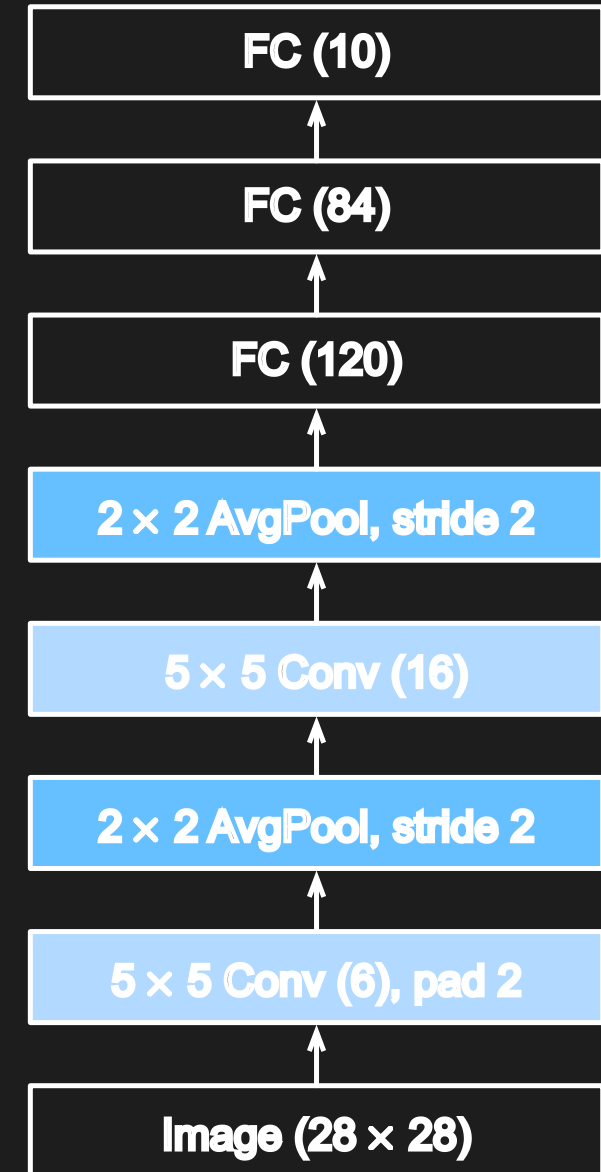
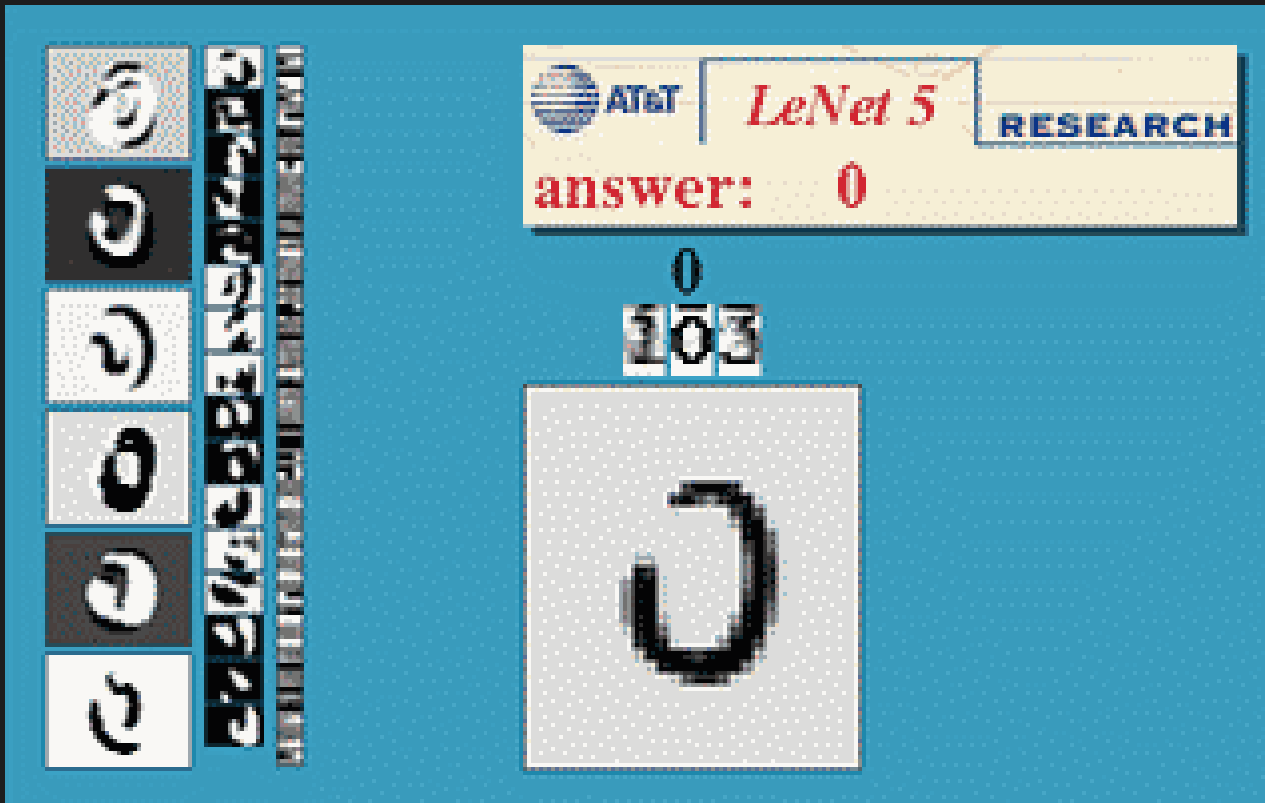


Pooling

- Again, a fixed-shape window sliding over the input matrix.
- Padding and strides also apply.
- No learnable parameters.
- Determine a value for each window.
 - Max pooling: max value in the window;
 - Avg. pooling: avg. value of the window.



LeNet (LeCun et al., 1998)

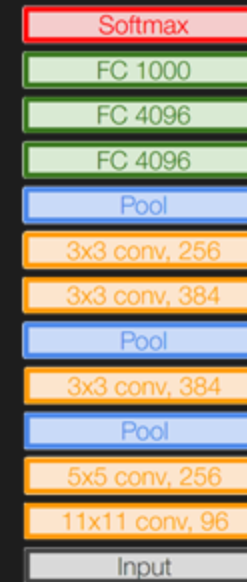
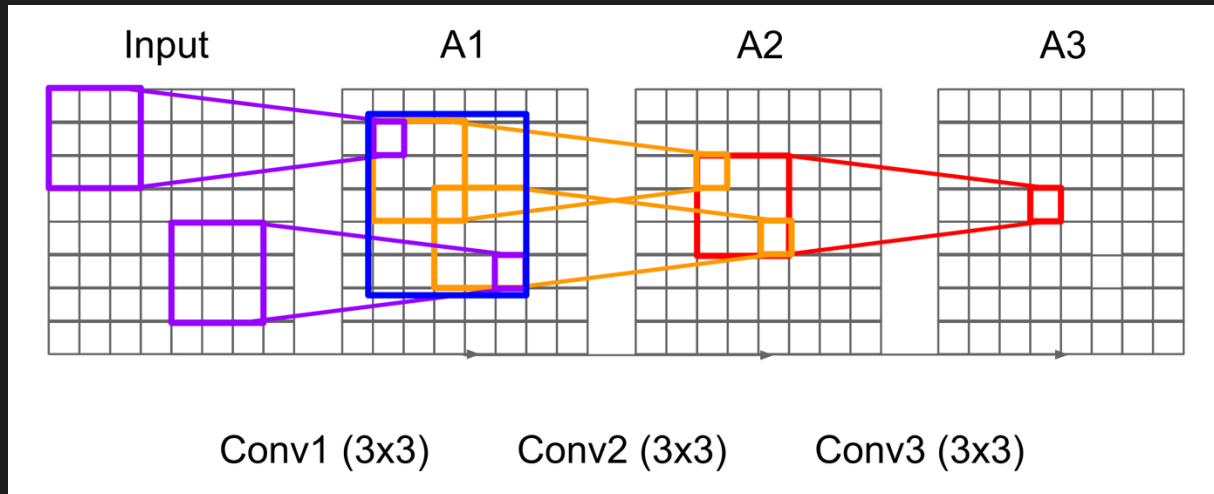


AlexNet (2012)

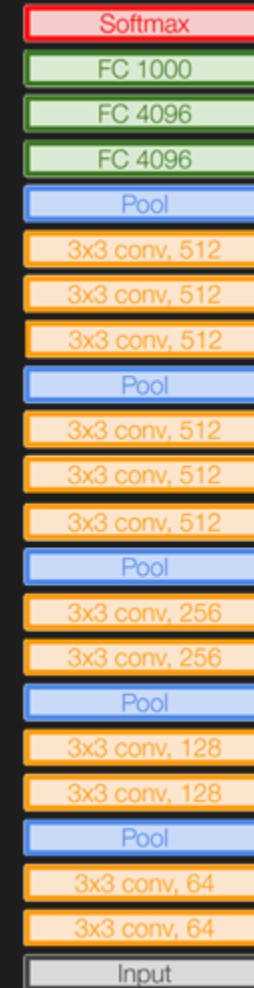


VGGNet (Simonyan et al., 2014)

- Small filters, deeper networks
 - 8 layers -> 16 – 19 layers
 - 3 x 3 conv
 - 2 x 2 max pooling



AlexNet



VGG16



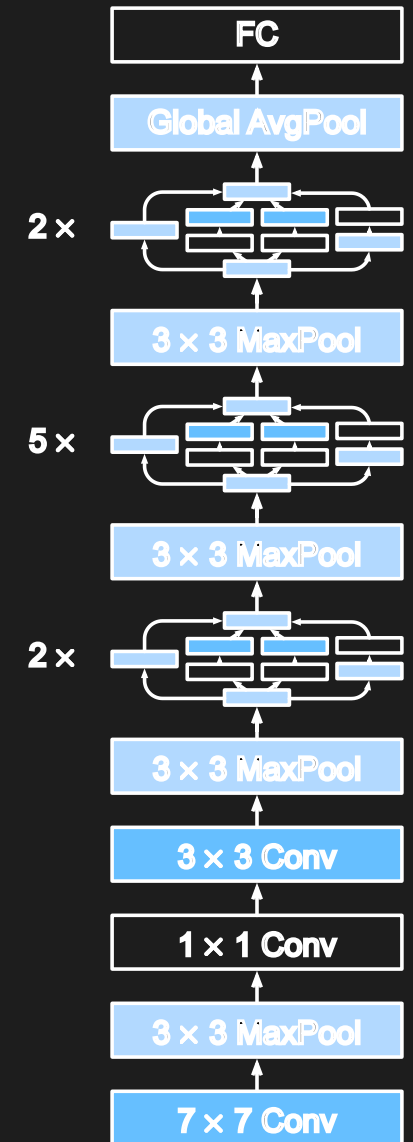
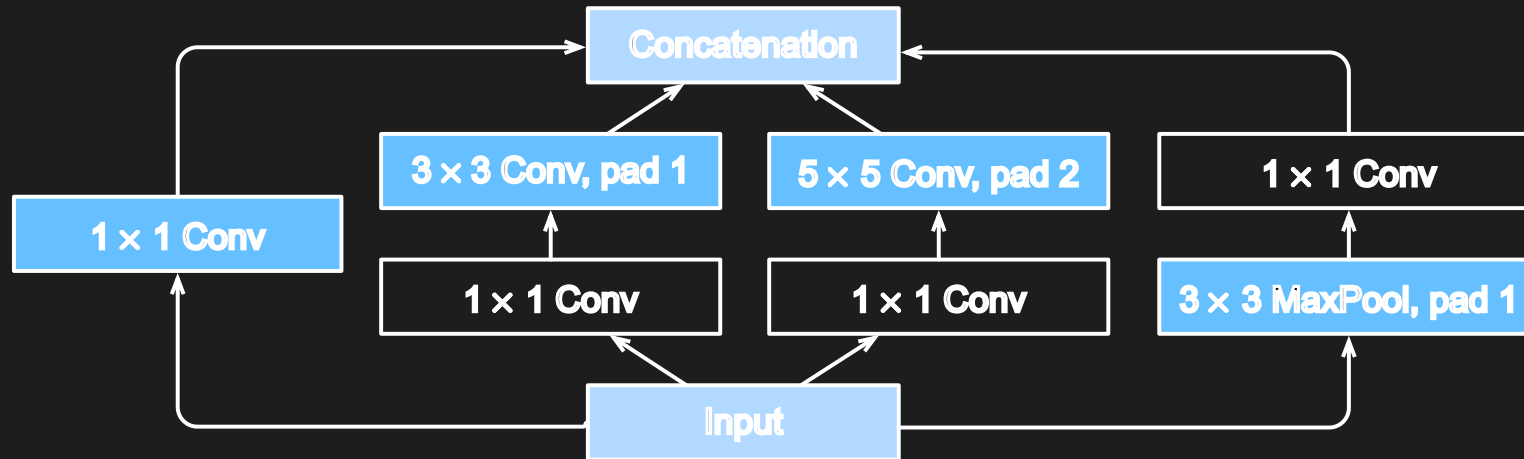
VGG19



GoogLeNet (Szegedy et al., 2014)

- Inception Blocks

- 1×1 conv + 3×3 conv + 5×5 conv to extract information from different spatial sizes.
- 1×1 conv on the input to reduce the number of channels.
- Concatenation along the channel dimension.

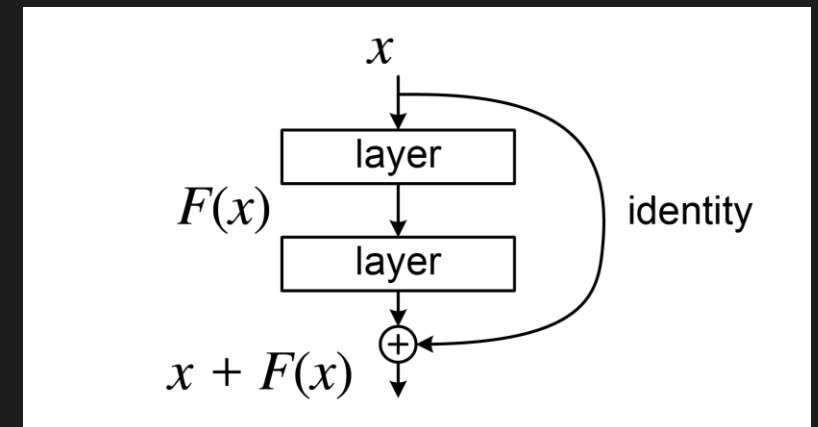


What is Inside in a Model?

Residual Connection & ResNet

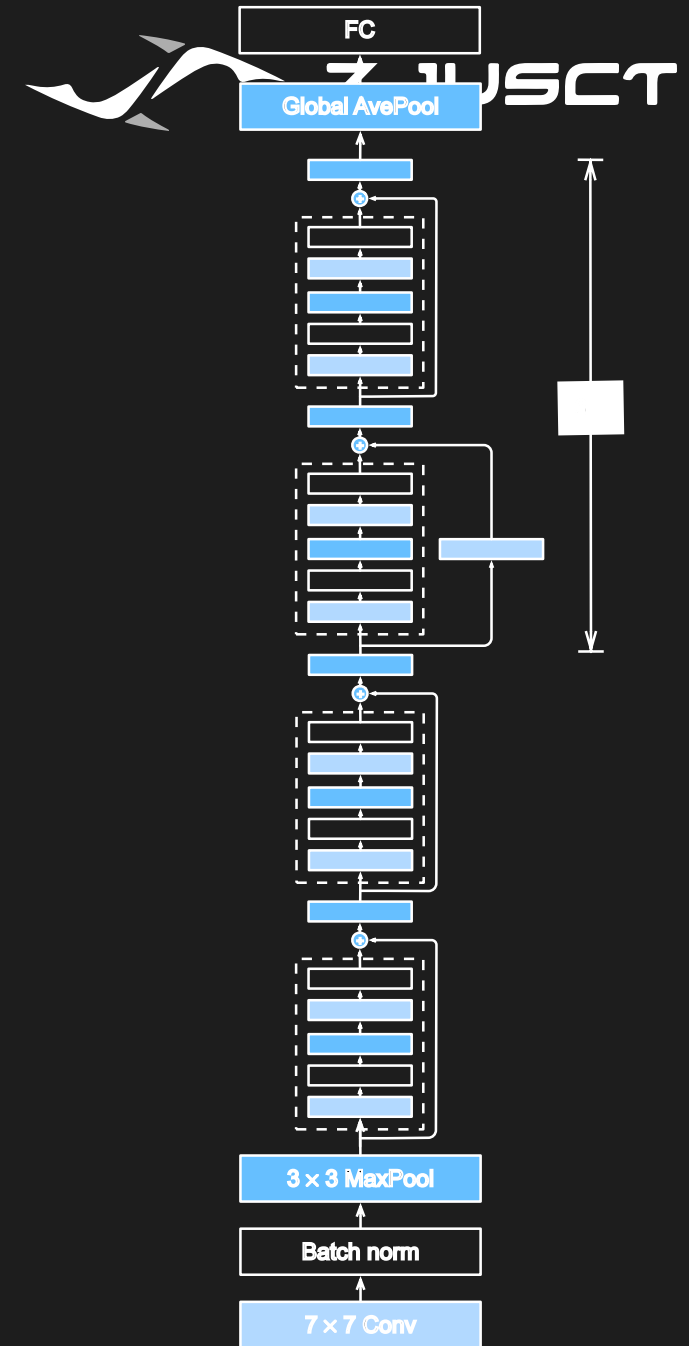
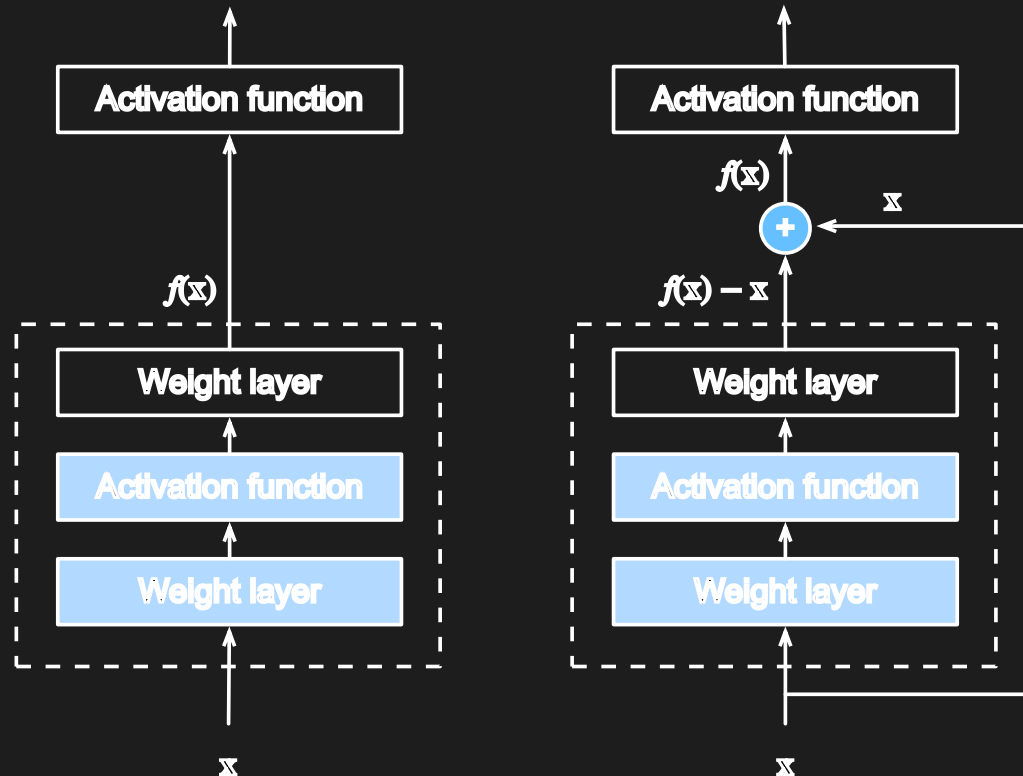
- A very common skill in neural network model
- Mitigate the vanishing gradient problem
- Make it possible to have deeper networks

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial x_\ell} &= \frac{\partial \mathcal{E}}{\partial x_L} \frac{\partial x_L}{\partial x_\ell} \\ &= \frac{\partial \mathcal{E}}{\partial x_L} \left(1 + \frac{\partial}{\partial x_\ell} \sum_{i=\ell}^{L-1} F(x_i) \right) \\ &= \frac{\partial \mathcal{E}}{\partial x_L} + \frac{\partial \mathcal{E}}{\partial x_L} \frac{\partial}{\partial x_\ell} \sum_{i=\ell}^{L-1} F(x_i)\end{aligned}$$



ResNet (He et al., 2016)

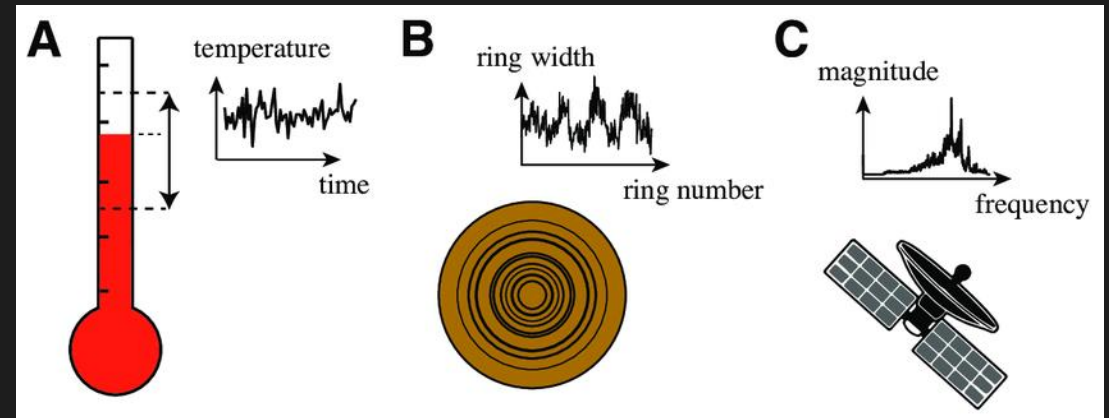
- Learn the residual mapping $f(\mathbf{x}) - \mathbf{x}$ instead.
- Ensure $\mathcal{F}_i \subseteq \mathcal{F}_{i+1}$ (let residual mapping = 0)



Recurrent Neural Network (RNN)

Sequential Data

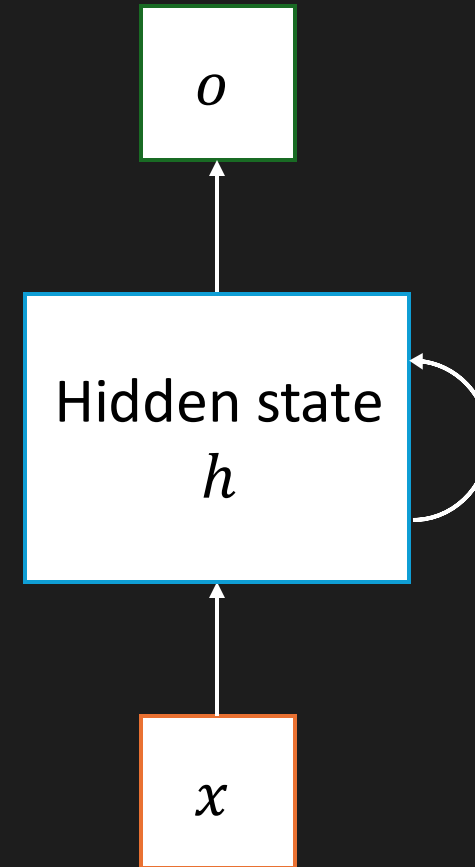
- Text
- Audio
- Stock quotes
- Video frames
- Motions and gestures
- Image as a sequence of pixels



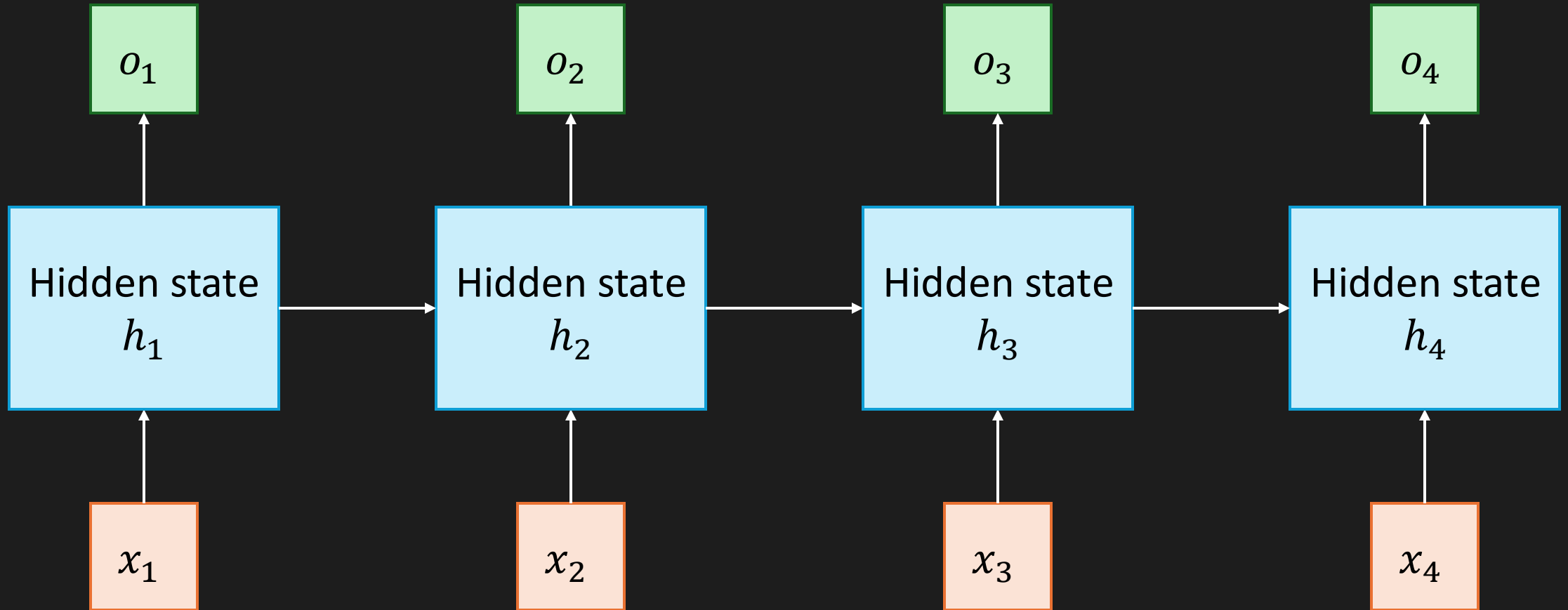
What is Inside in a Model?

Recurrent Neural Network

- Hidden state “remembers” sequential input.
- For each step, update hidden state and determine output using recurrence formula:
 - $h_t = f(h_{t-1}, x_t; w_h)$
 - $o_t = g(h_t; w_o)$



Unfolding Calculation Graph

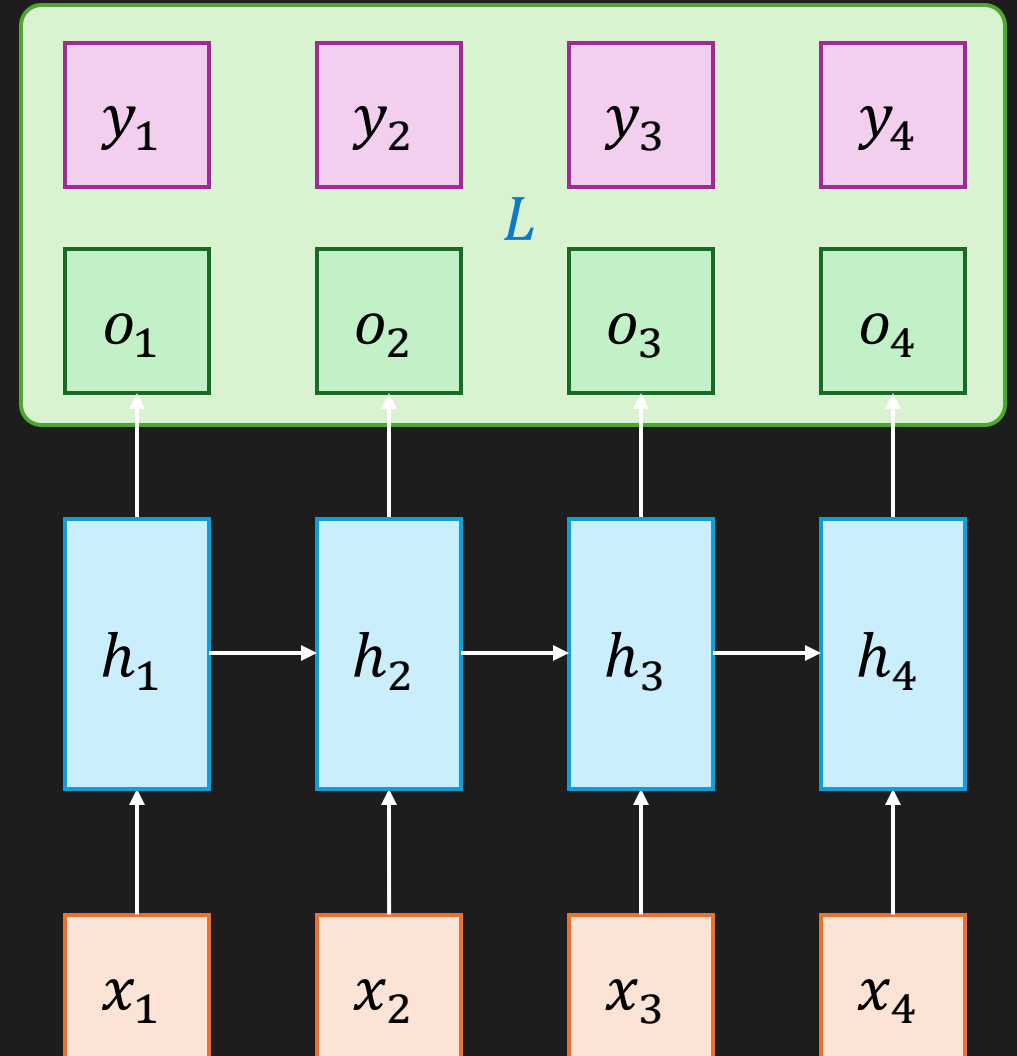


Input Output Relationship



Backpropagation Through Time

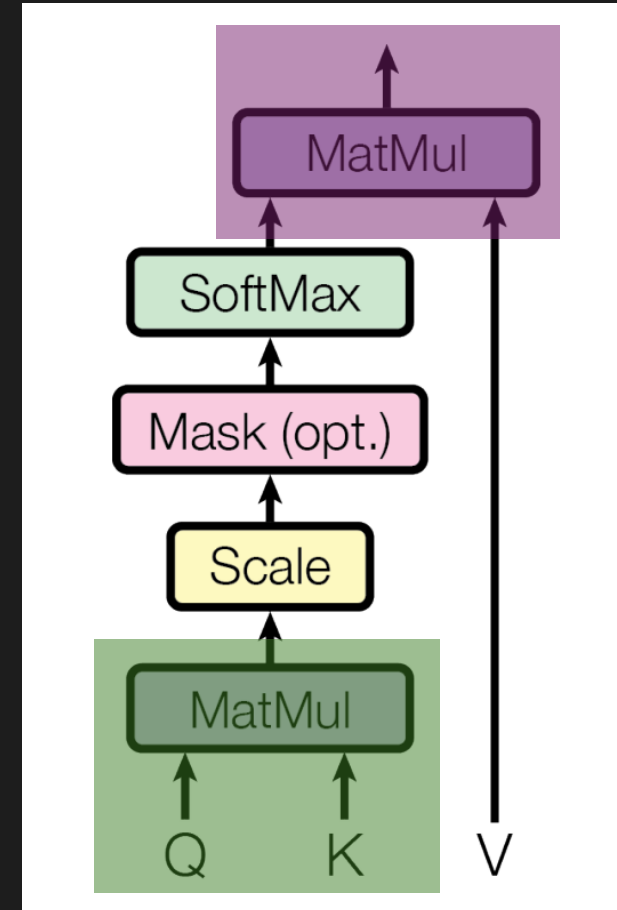
- $h_t = f(h_{t-1}, x_t; w_h)$
- $o_t = g(h_t; w_o)$
- $L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t).$
- $\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$



Attention & Transformer

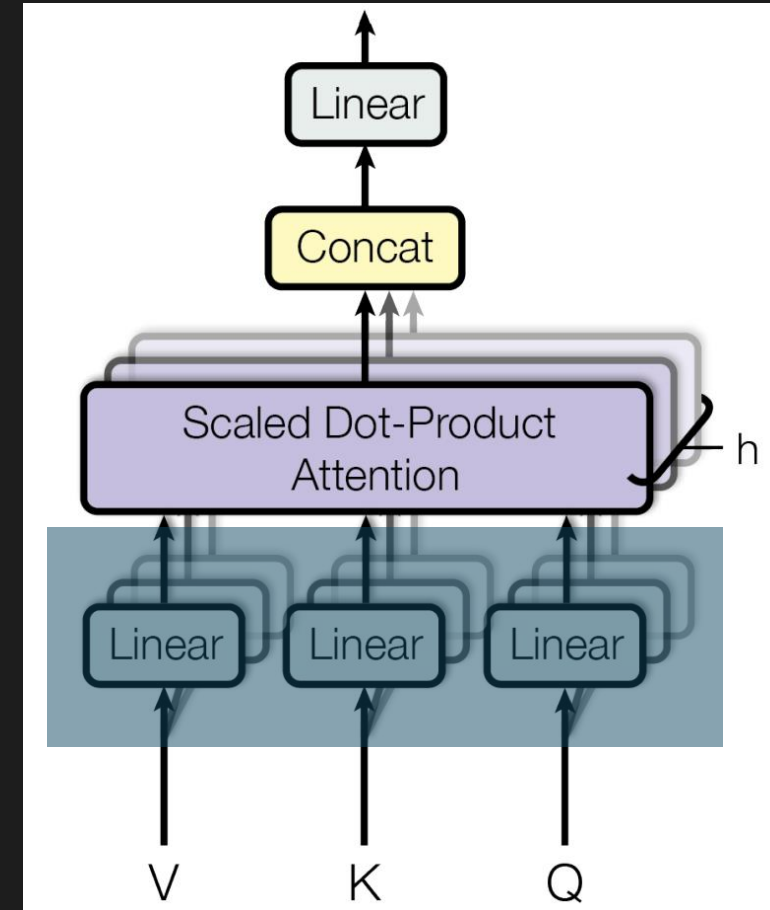
Scaled Dot-Product Attention

- $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$



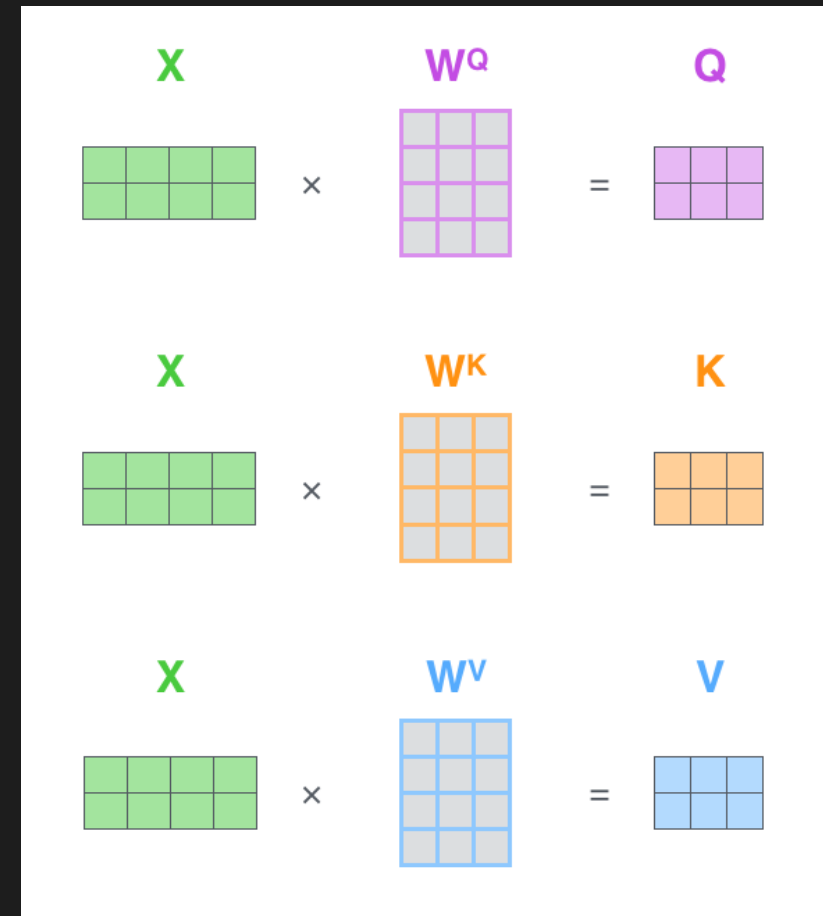
Multi-Head Attention

- $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$
- Where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$
- Linear projection for Q, K, V
 - Add more expressivity
- Different linear layer for different head
 - Let heads care about different things

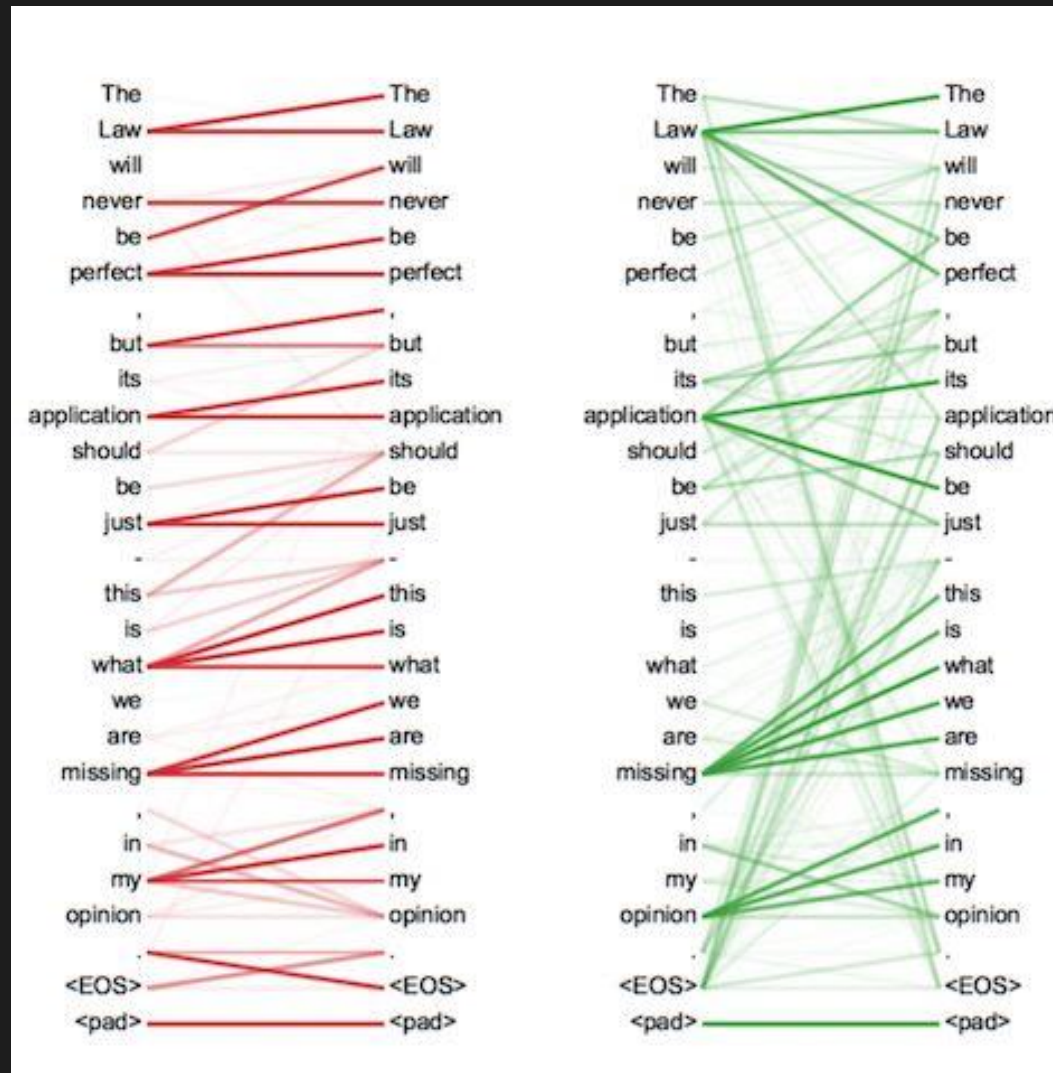


Self-Attention

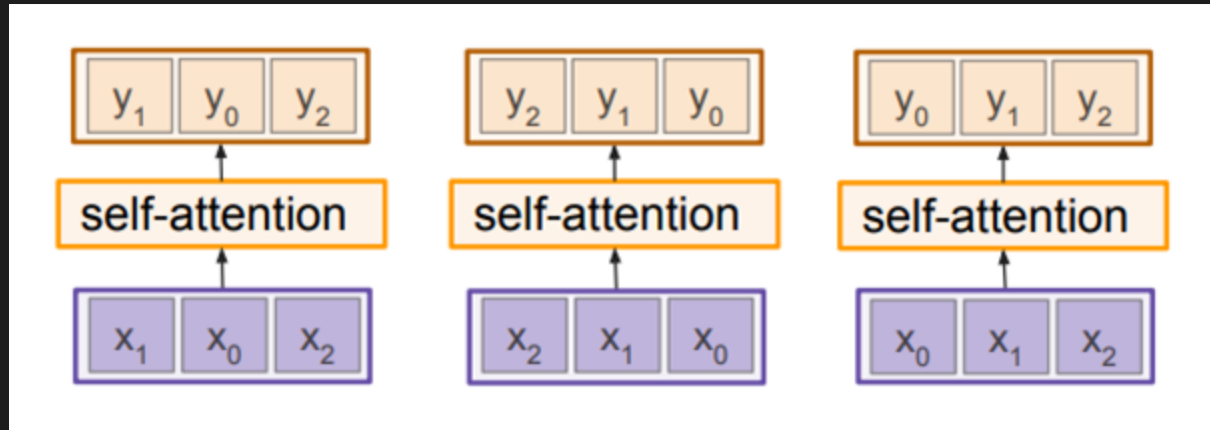
- $V = K = Q = X$
- $\text{Attention}(X, X, X)$
- Q : meaning behind?



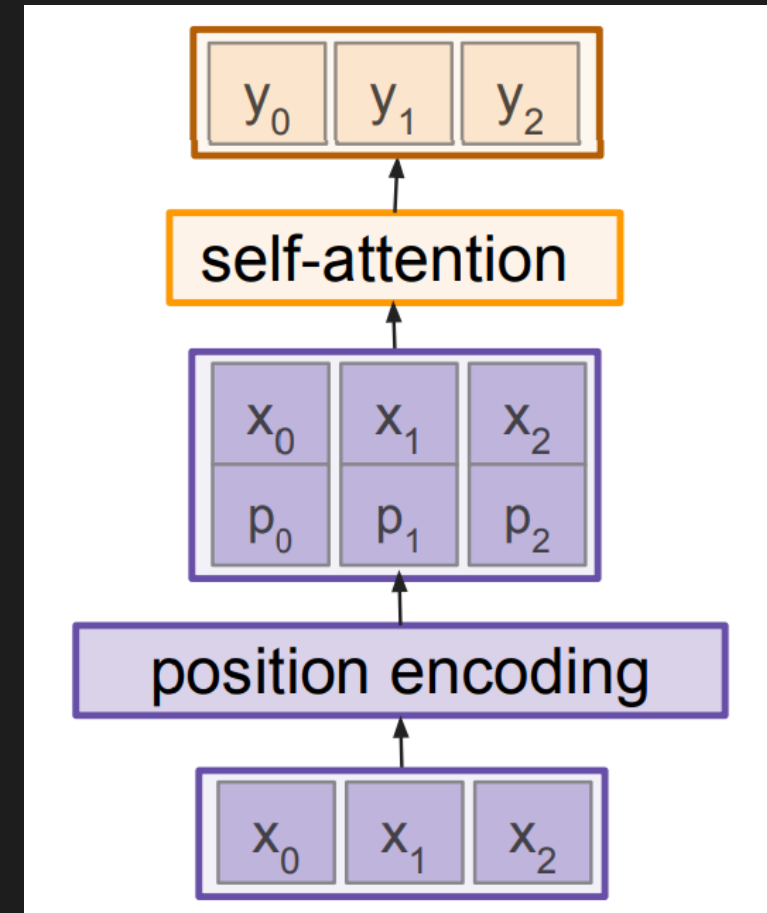
Visualize Self-Attention



Permutation Equivariant Problem in Self-Attention

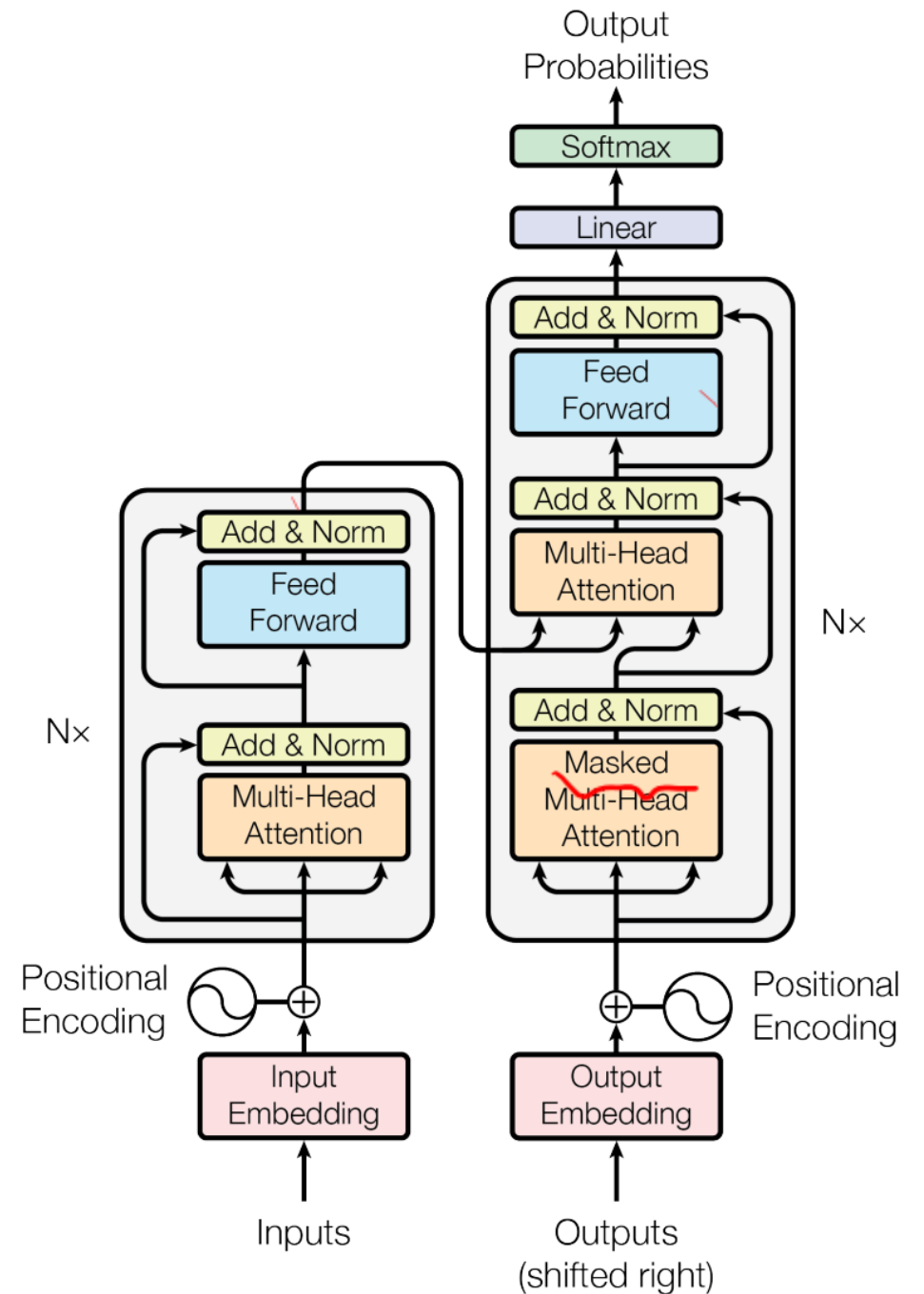


- Solution: Add position actively
- Position encoding
 - sin-cos
 - Learned encoding

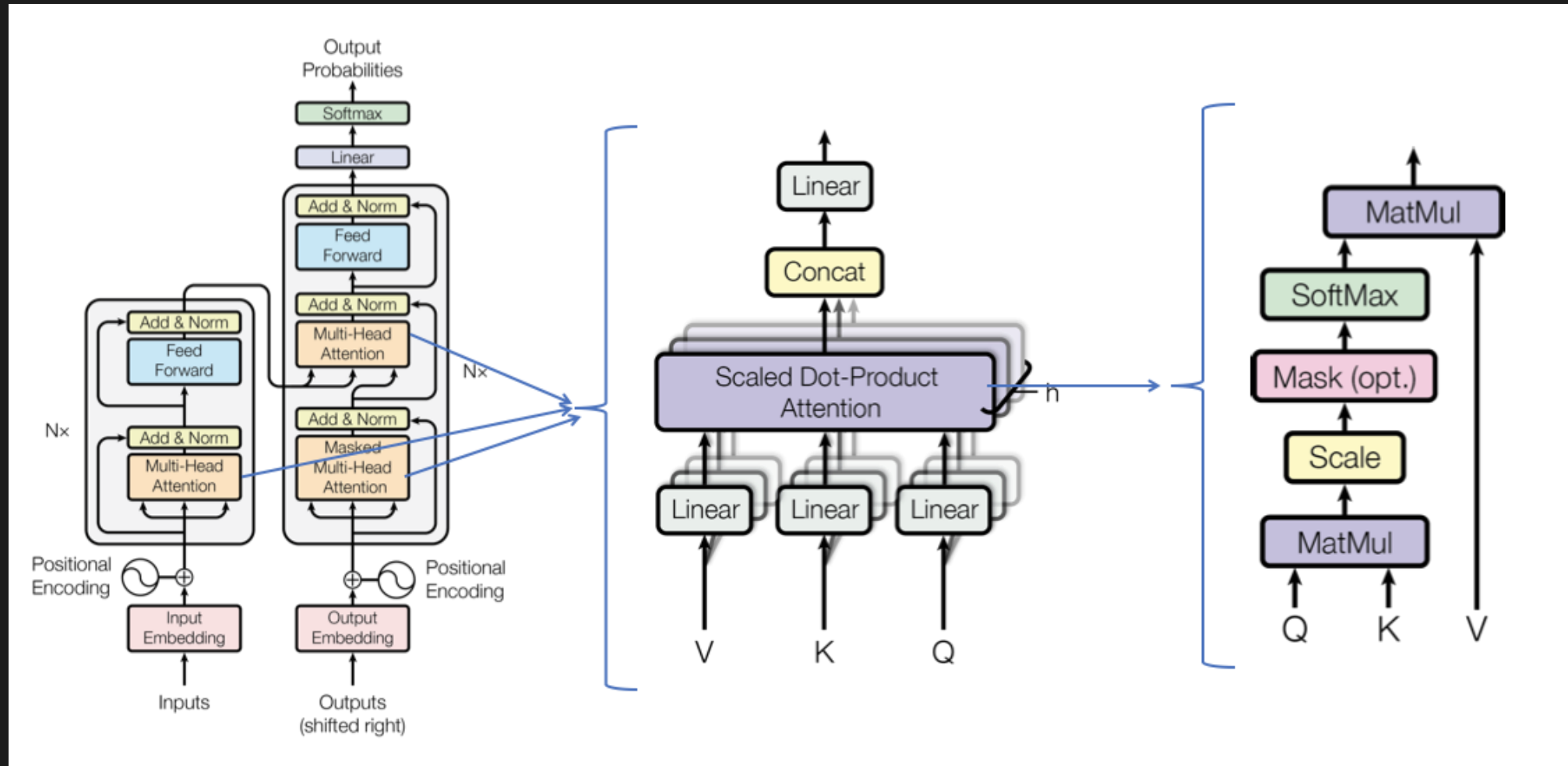


Transformer (Vaswani et al., 2017)

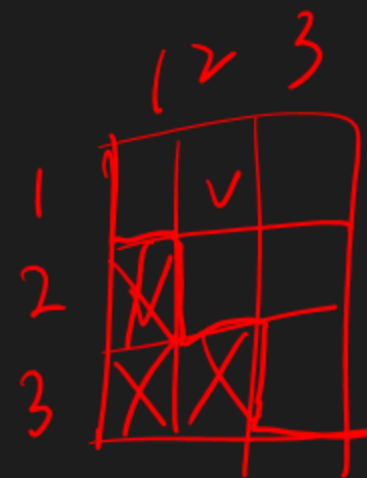
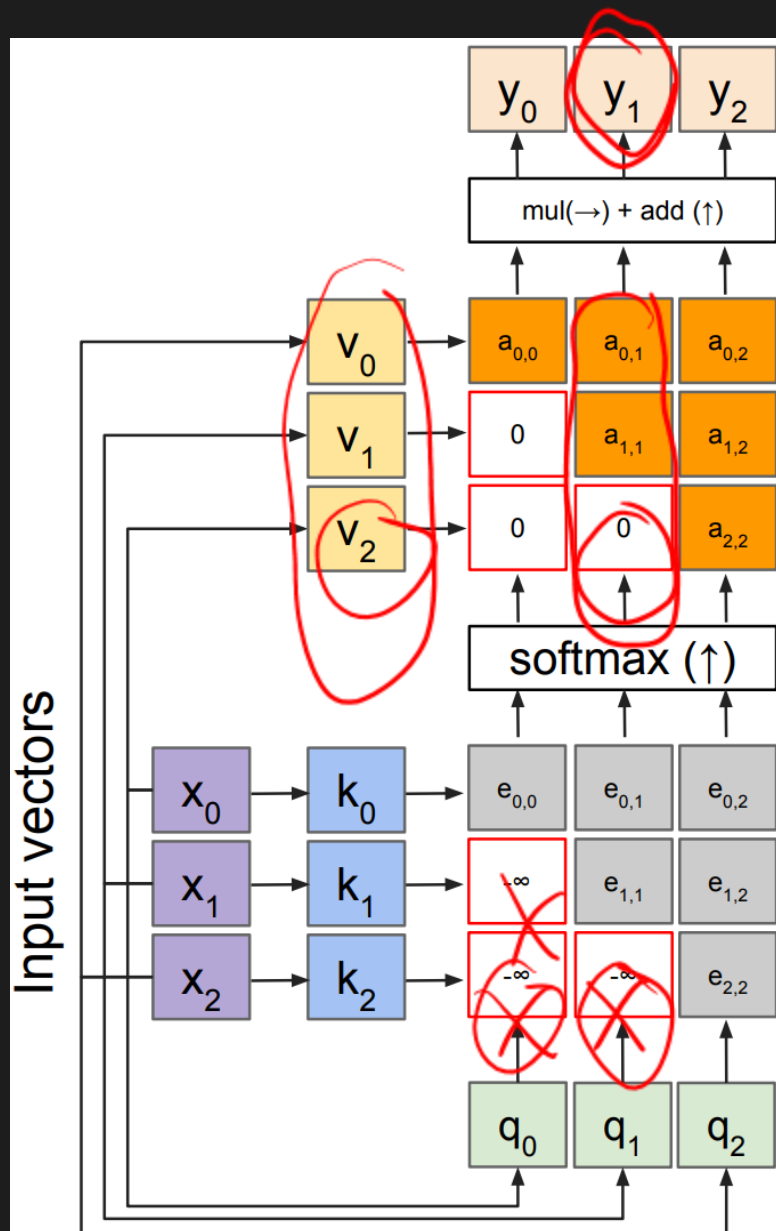
- Encoder-Decoder
- Masked MHA
 - Prevent vectors from looking at future vectors when training



Transformer (Vaswani et al., 2017)

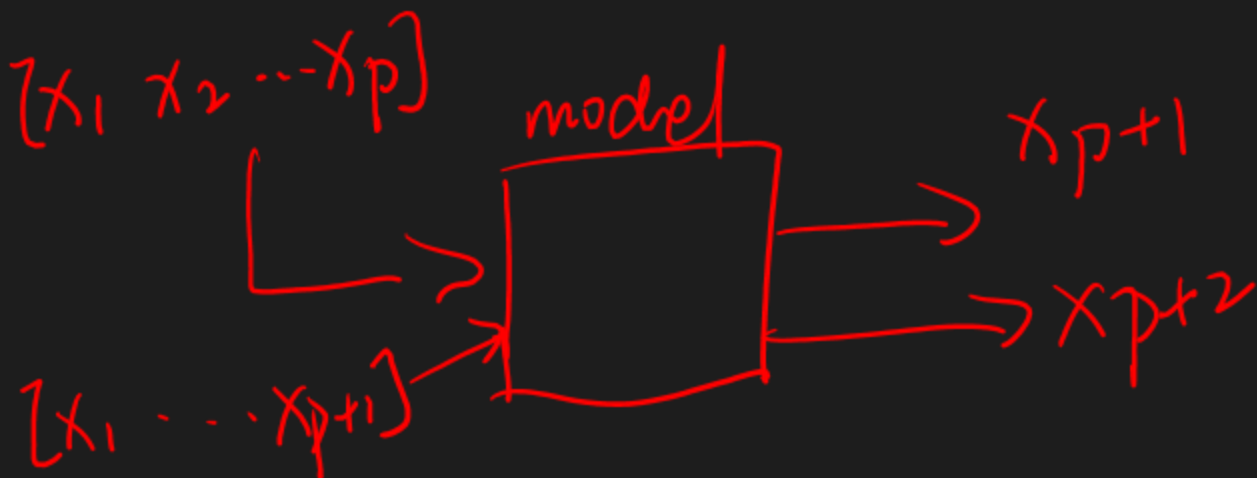


Masked MHA



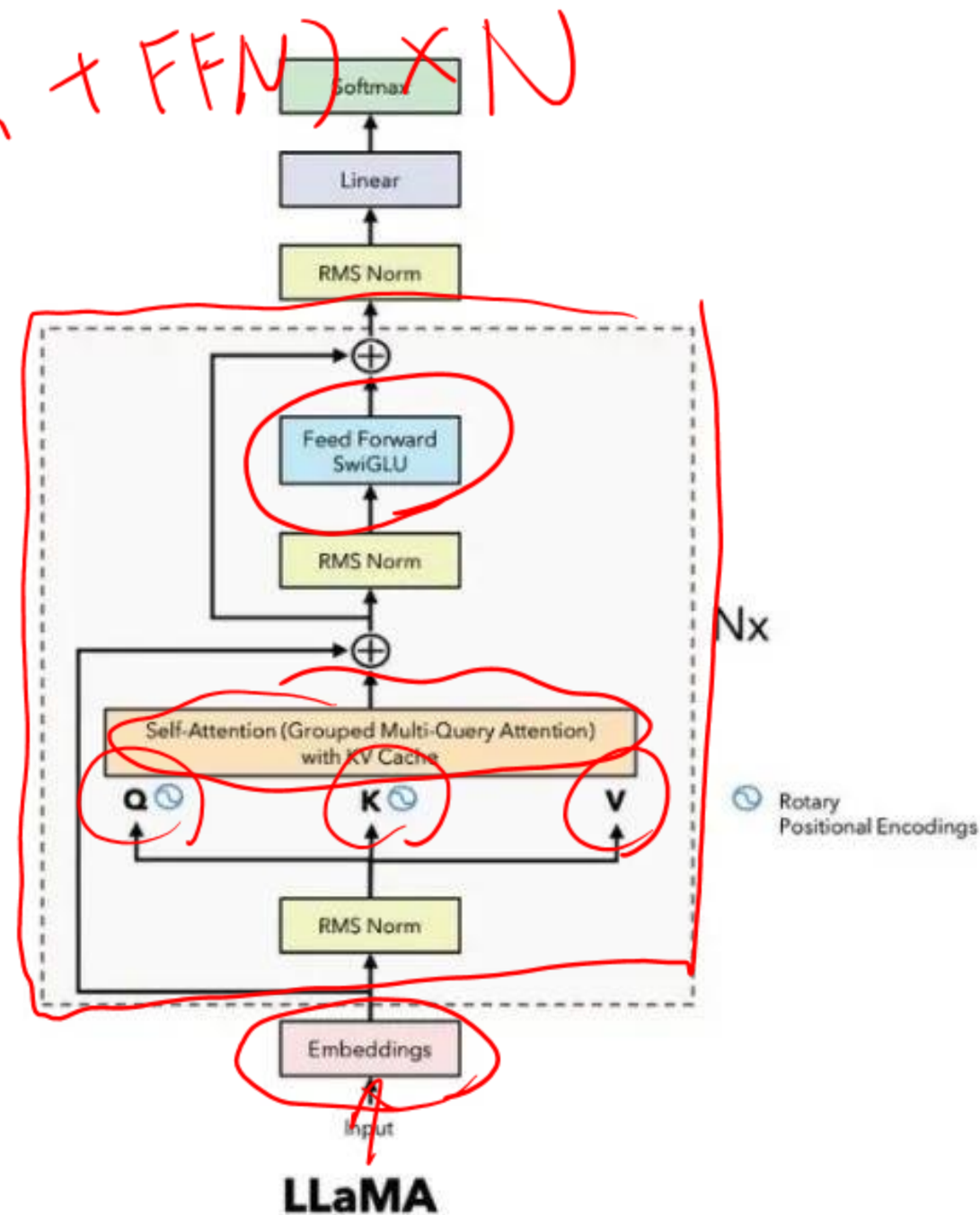
Llama (Hugo et al., 2023)

- Decoder only
- QK encoded with RoPE
- RMSNorm vs LayerNorm
- SwiGLU vs ReLU



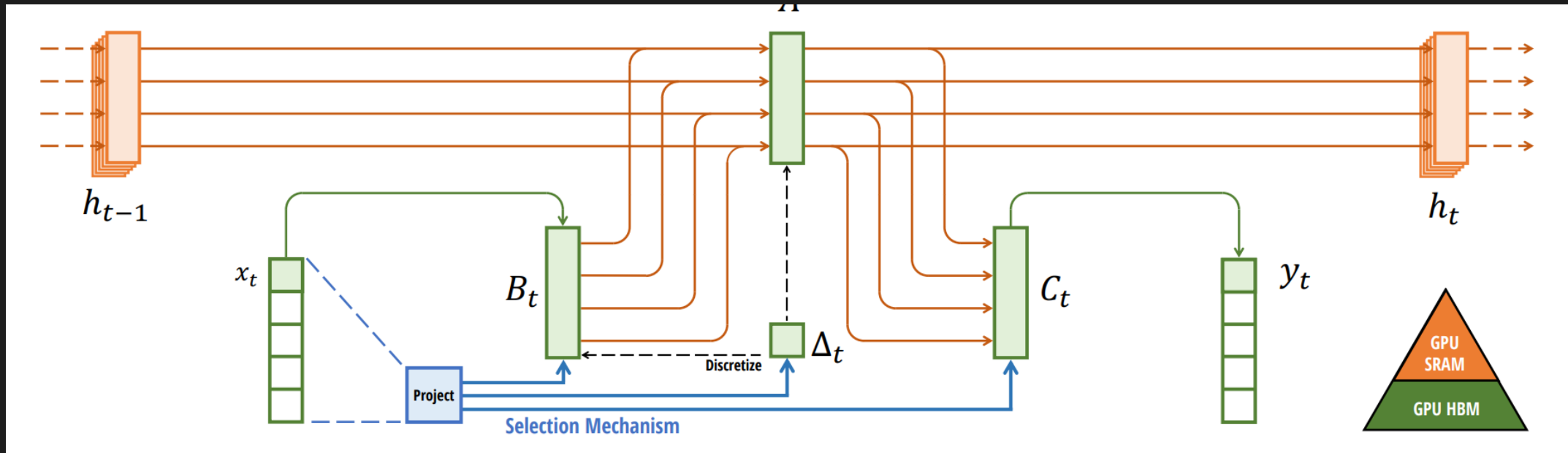
Attention + FFN) x N

MLA



Mamba (Albert et al., 2023)

- More Efficient Model Architecture
- Based on SSM: $x(t) \rightarrow y(t)$ with $h(t)$

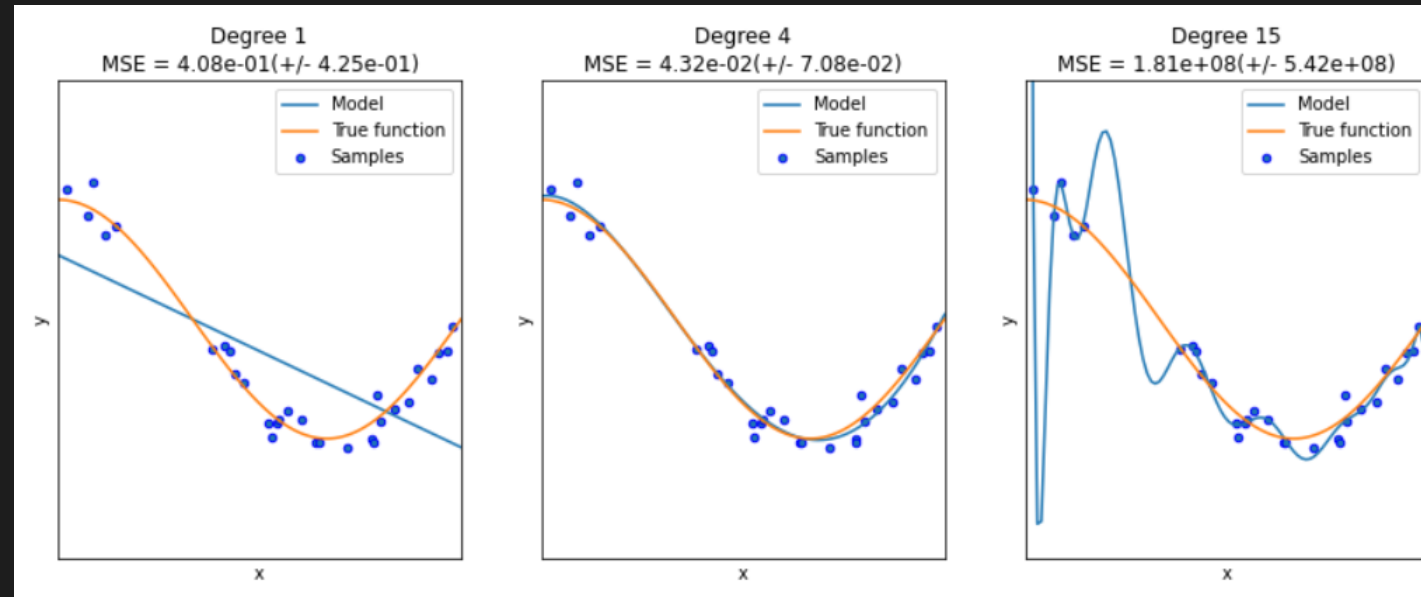


Any Questions?

What is Inside in a Model?

Normalization & Regularization

- How to solve over-fitting problem?
 - Make model easier

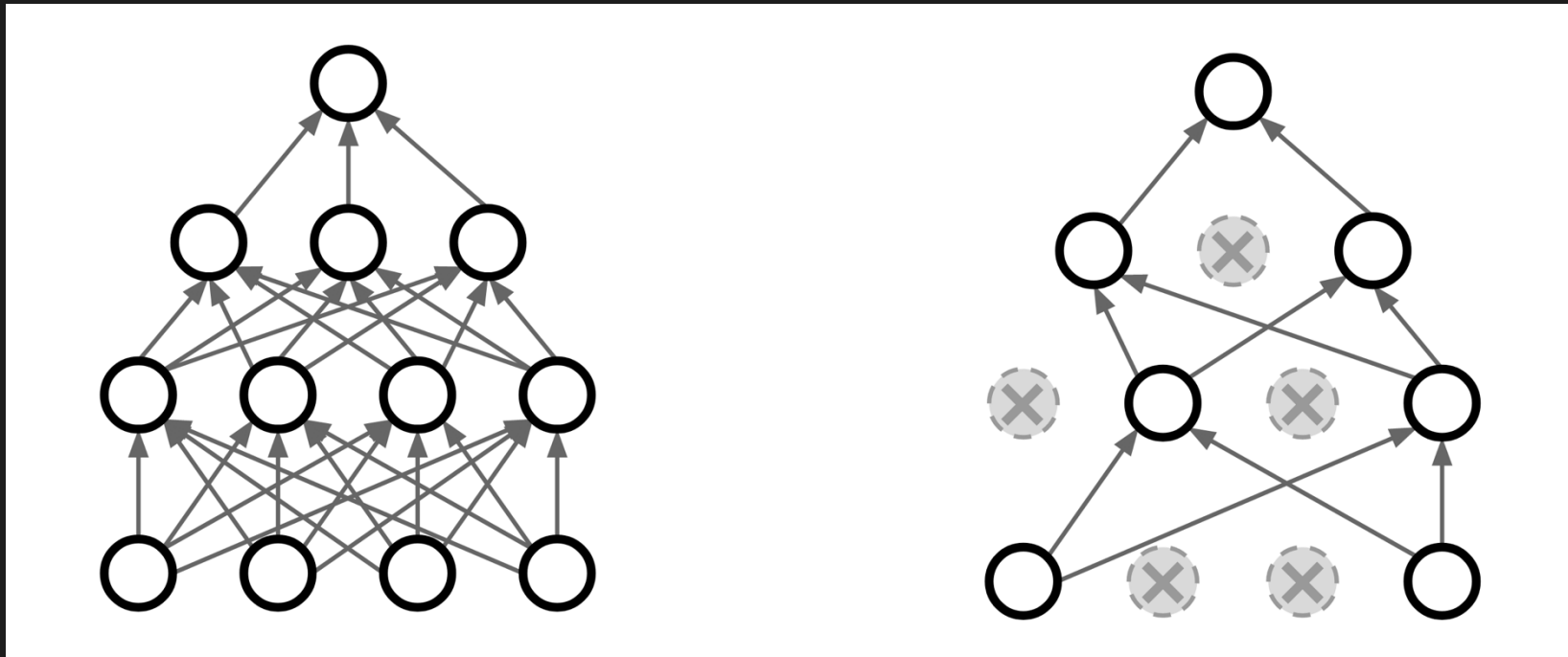


Regularization

- Regularization term
 - $L'(W) = L(W) + \lambda R(W)$
 - L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$
 - L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

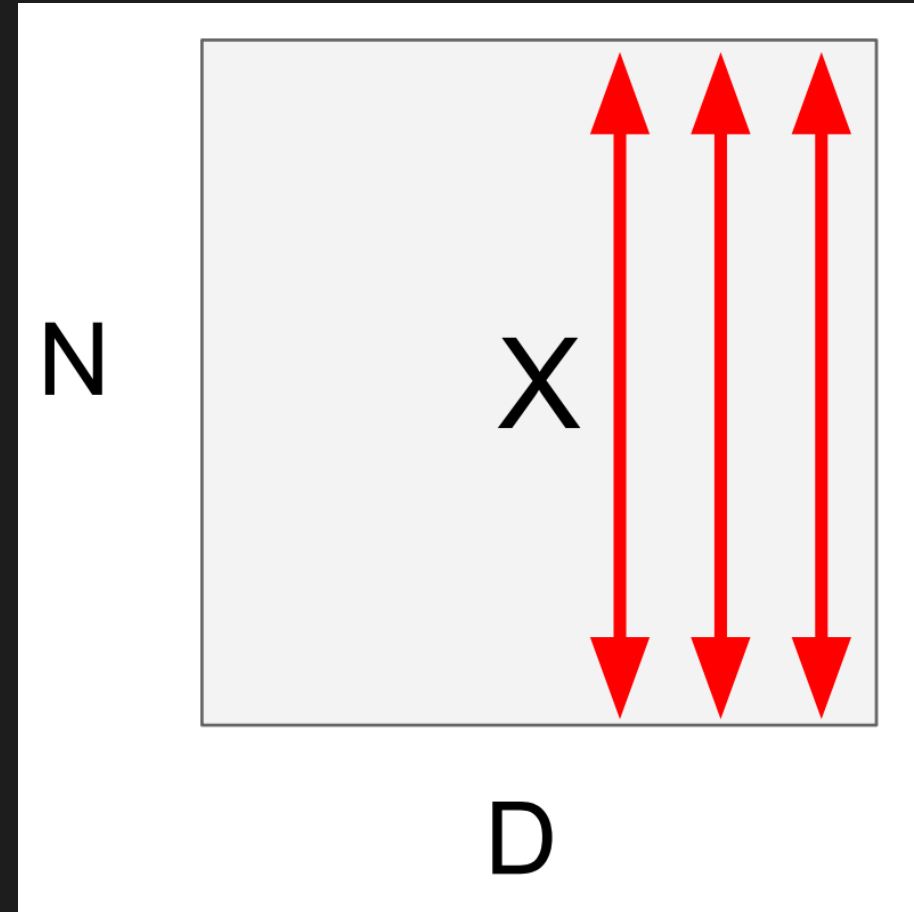
Dropout (Srivastava et al., 2014)

- In each forward pass, randomly set some neurons to zero
- Probability of dropping is a hyperparameter



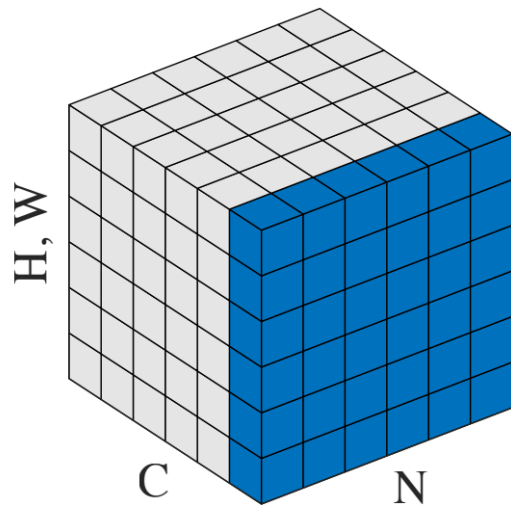
Batch Normalization

- Motivation: input x can be bad in distribution
 - Not centered around 0
 - Different scaling at each element
- Input: x $N \times D$
- Mean: $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$ D
- Var: $\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$ D
- Output: $\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$ $N \times D$
- Q: any information loss?

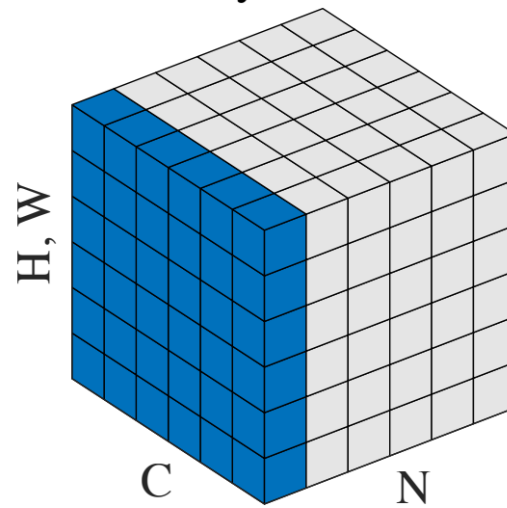


Normalization Methods

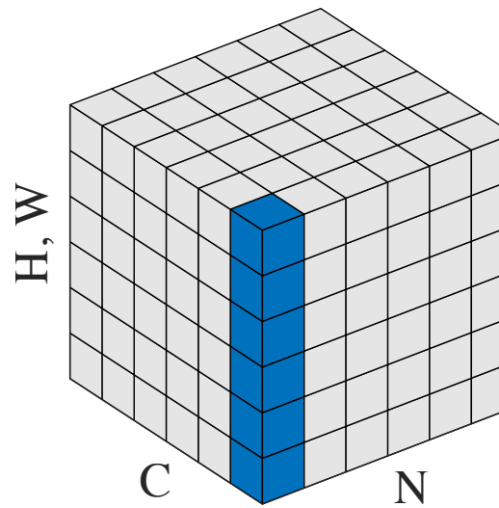
Batch Norm



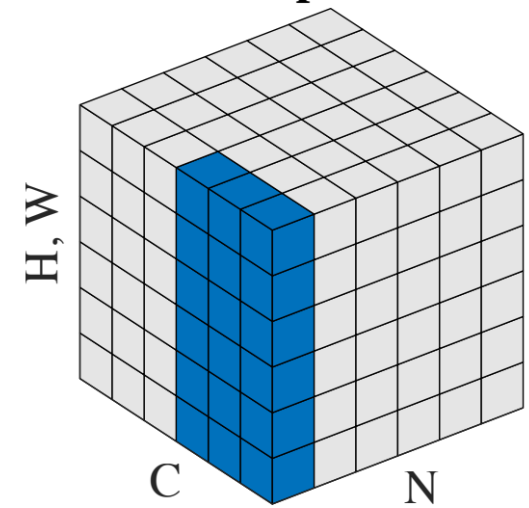
Layer Norm



Instance Norm



Group Norm



Any Questions?

Practice: LeNet Training